

Advanced Machine Learning

Lecture 3: Variants of SGD & Regularization

Jens-Peter M. Zemke
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

29 October 2024



Outline

Repetition

- Feedforward
- Backpropagation

Initialization

- Zero/known/random weights
- Xavier Glorot, Kaiming He, ...

Heuristic variants of (S)GD

- Momentum & NAG
- Adaptive learning rates (Adagrad, RMSprop)
- Combinations (Adam, Nadam)

Regularization

- L^2 and L^1 regularization
- Batch normalization
- Dropout & DropConnect

Outline

Repetition

Feedforward

Backpropagation

Initialization

Zero/known/random weights

Xavier Glorot, Kaiming He, ...

Heuristic variants of (S)GD

Momentum & NAG

Adaptive learning rates (Adagrad, RMSprop)

Combinations (Adam, Nadam)

Regularization

L^2 and L^1 regularization

Batch normalization

Dropout & DropConnect

Artificial Neural Network $\rightsquigarrow$ Feedforward Neural Network, FNN

Known ANN uniquely defined by

- ▶ number m of layers
- ▶ number of neurons per layer, uniquely coded by $\mathbf{n} \in \mathbb{N}^m$
- ▶ **parameters**: weight matrices $\mathbf{W}_i \in \mathbb{R}^{n_{i+1} \times n_i}$, bias vectors $\mathbf{b}_i \in \mathbb{R}^{n_{i+1}}$, activation functions a_i , $i = 1, \dots, m-1$

Name “Feedforward” due to **layerwise forward** way of evaluation of function $\mathbf{f} : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_m}$:

$$\mathbf{a}_1 := \mathbf{x}, \quad \mathbf{z}_i := \mathbf{W}_i \mathbf{a}_i + \mathbf{b}_i, \quad \mathbf{a}_{i+1} := a_i(\mathbf{z}_i), \quad i = 1, \dots, m-1, \quad \mathbf{f}(\mathbf{x}) := \mathbf{a}_m.$$

Exercises:

- ▶ Current: implement a class `FeedforwardNet` in Python ✓
- ▶ Future: add methods `backprop()` & `train()`

Backpropagation

Theorem (Partial derivatives of ANN, part 1/2)

ANN & local cost function given,

$$C(\mathbf{p}) := \frac{1}{2} \|\mathbf{a}_m - \mathbf{y}\|_2^2, \quad \mathbf{a}_m = \mathbf{f}(\mathbf{x}).$$

Define for $i = 1, \dots, m-1$ the vectors

$$\boldsymbol{\delta}_i := \frac{\partial C}{\partial \mathbf{z}_i} := \begin{pmatrix} \frac{\partial C}{\partial z_{1,i}} \\ \vdots \\ \frac{\partial C}{\partial z_{n_{i+1},i}} \end{pmatrix} = \nabla_{\mathbf{z}_i} C.$$

Vectors $\boldsymbol{\delta}_i$ can be computed recursively *from last layer to first*,

$$\boldsymbol{\delta}_{m-1} = a'_{m-1}(\mathbf{z}_{m-1}) \circ (\mathbf{a}_m - \mathbf{y}), \quad \boldsymbol{\delta}_i = a'_i(\mathbf{z}_i) \circ (\mathbf{W}_{i+1}^\top \boldsymbol{\delta}_{i+1}), \quad i = m-2, m-3, \dots, 1.$$

Backpropagation

Theorem (Partial derivatives of ANN, part 2/2)

Derivatives of C with respect to the weights $\mathbf{W}_i$ and the biases $\mathbf{b}_i$, $i = 1, \dots, m - 1$, are given by

$$\frac{\partial C}{\partial \mathbf{W}_i} = \delta_i \mathbf{a}_i^\top, \quad \frac{\partial C}{\partial \mathbf{b}_i} = \delta_i.$$

Backpropagation

Theorem (Partial derivatives of ANN, part 2/2)

Derivatives of C with respect to the weights $\mathbf{W}_i$ and the biases $\mathbf{b}_i$, $i = 1, \dots, m - 1$, are given by

$$\frac{\partial C}{\partial \mathbf{W}_i} = \delta_i \mathbf{a}_i^\top, \quad \frac{\partial C}{\partial \mathbf{b}_i} = \delta_i.$$

Implementation of minibatches:

- ▶ **simultaneous evaluation** $\rightsquigarrow$ matrix-matrix multiplications $\mathbf{Z}_i = \mathbf{W}_i \mathbf{A}_i + \mathbf{b}_i \mathbf{e}^\top$, initialization $\mathbf{A}_1 = (\mathbf{x}_1, \dots, \mathbf{x}_k)$, $\mathbf{e} \in \mathbb{R}^k$ vector comprising ones
- ▶ evaluation of **activation functions elementwise**, $\mathbf{A}_{i+1} = a_i(\mathbf{Z}_i)$
- ▶ **backpropagation with matrices** $\rightsquigarrow$ derivative of errors $(\mathbf{A}_m - (\mathbf{y}_1, \dots, \mathbf{y}_k))/k$, all scaled $\delta_i^{(1)}, \dots, \delta_i^{(k)}$ terms simultaneously using matrix-matrix multiplications as matrices Δ_i
- ▶ **averaged updates** are $\Delta_i \mathbf{A}_i^\top$ and $\Delta_i \mathbf{e}$

Differences between math & implementation: $\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$

Suppose that we have f input features, d inputs, n neurons in the considered layer

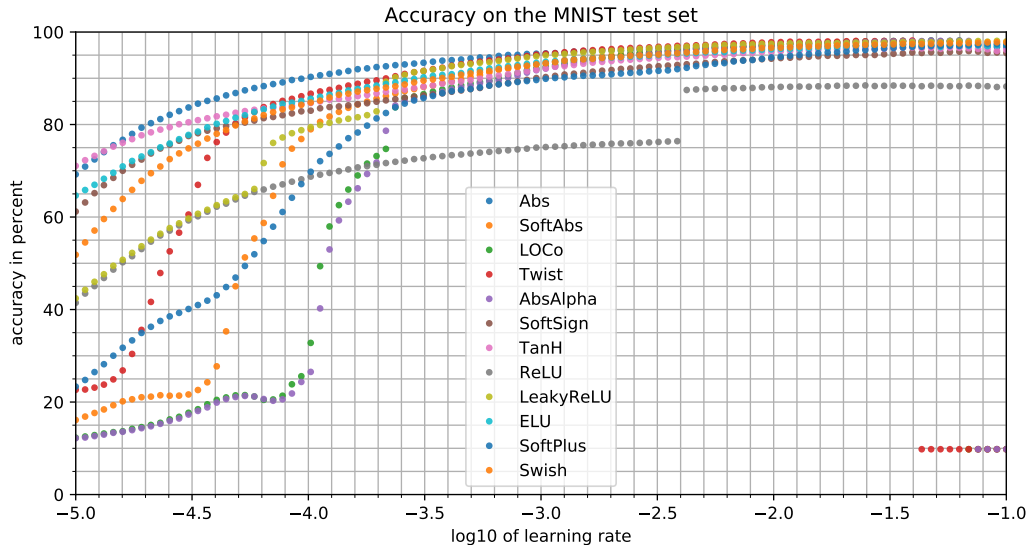
Quantity	Math/NumPy	TF/Keras
data $\mathbf{a}$	$\mathbf{a} \in \mathbb{R}^{f \times d}$	$\mathbf{a} \in \mathbb{R}^{d \times f}$
weight $\mathbf{W}$	$\mathbf{W} \in \mathbb{R}^{n \times f}$	$\mathbf{W} \in \mathbb{R}^{f \times n}$
bias $\mathbf{b}$	$\mathbf{b} \in \mathbb{R}^{n \times 1}$	$\mathbf{b} \in \mathbb{R}^{1 \times n}$
feedforward ¹	$\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b} \in \mathbb{R}^{n \times d}$	$\mathbf{z} = \mathbf{a}\mathbf{W} + \mathbf{b} \in \mathbb{R}^{d \times n}$
delta $\delta = \frac{\partial C}{\partial \mathbf{z}}$	delta = $\delta \in \mathbb{R}^{n \times d}$	delta = $\delta \in \mathbb{R}^{d \times n}$
update $\frac{\partial C}{\partial \mathbf{W}}$	$d\mathbf{W} = \delta \mathbf{a}^T / d \in \mathbb{R}^{n \times f}$	$d\mathbf{W} = \mathbf{a}^T \delta / d \in \mathbb{R}^{f \times n}$
update ² $\frac{\partial C}{\partial \mathbf{b}}$	$d\mathbf{b} = \delta \mathbf{e} / d \in \mathbb{R}^{n \times 1}$	$d\mathbf{b} = \mathbf{e}^T \delta / d \in \mathbb{R}^{1 \times n}$
backprop $\frac{\partial C}{\partial \mathbf{a}}$	$\mathbf{W}^T \delta \in \mathbb{R}^{f \times d}$	$\delta \mathbf{W}^T \in \mathbb{R}^{d \times f}$

Table: Math/NumPy versus TF/Keras

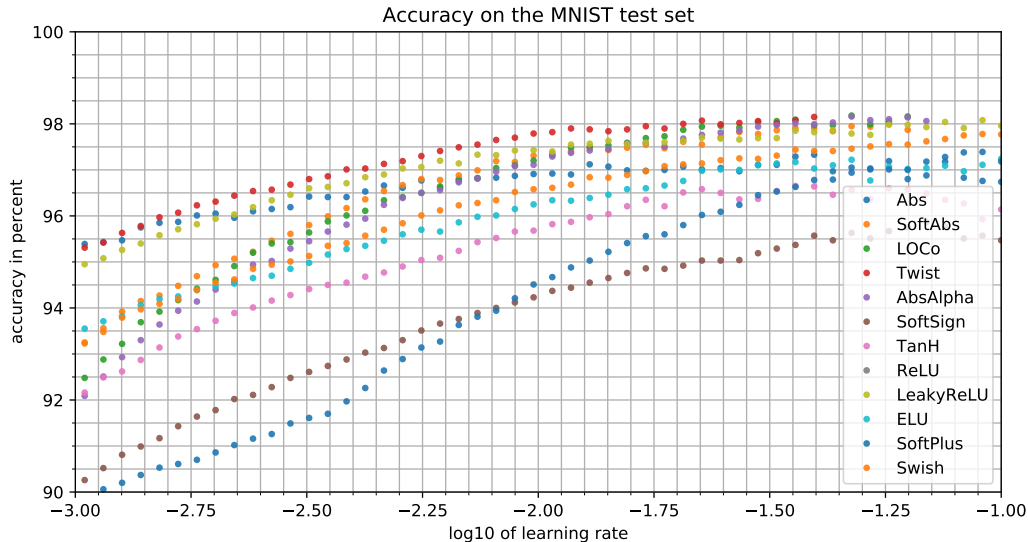
¹Broadcasting in Python, $\mathbf{b}$ is added to all columns or rows

²Sum using $\mathbf{e} \in \mathbb{R}^{d \times 1}$, the vector of all ones

One hidden layer on MNIST



One hidden layer on MNIST



Outline

Repetition

- Feedforward
- Backpropagation

Initialization

- Zero/known/random weights
- Xavier Glorot, Kaiming He, ...

Heuristic variants of (S)GD

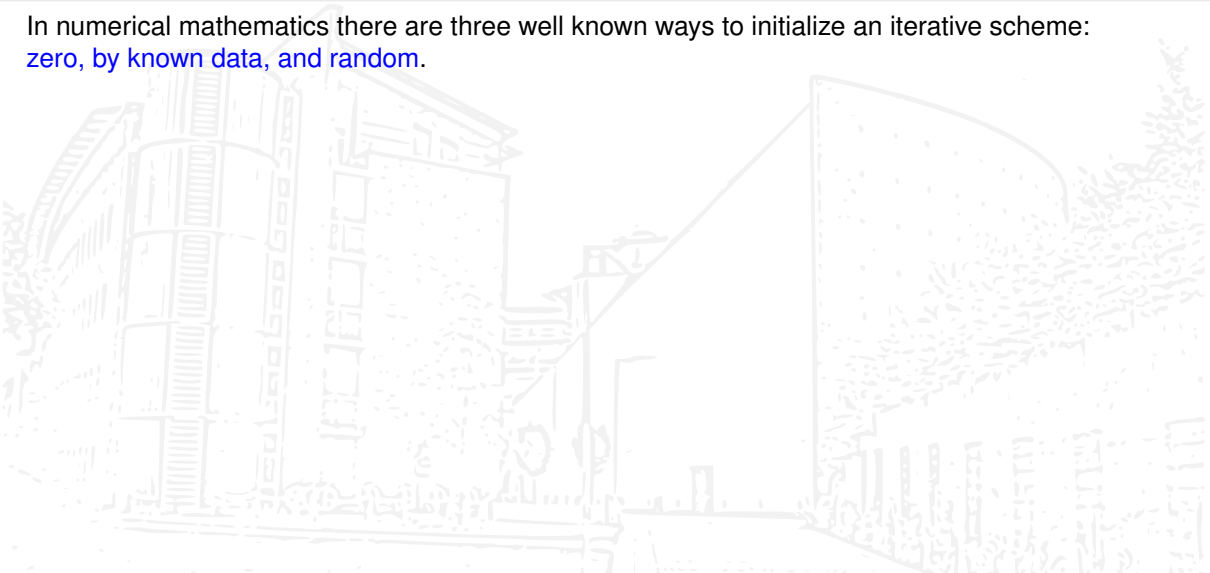
- Momentum & NAG
- Adaptive learning rates (Adagrad, RMSprop)
- Combinations (Adam, Nadam)

Regularization

- L^2 and L^1 regularization
- Batch normalization
- Dropout & DropConnect

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, by known data, and random.



Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, **by known data**, and **random**.

Initialize weights and biases with zero:

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, **by known data**, and **random**.

Initialize weights and biases with zero:

- ▶ 1st feedforward: $\mathbf{a}_1 = \mathbf{x}$, $\mathbf{z}_i = \mathbf{O}\mathbf{a}_i + \mathbf{o} = \mathbf{o}$, $\mathbf{a}_{i+1} = a_i(\mathbf{z}) = a_i(\mathbf{o})$ **constant**, $i = 1, \dots, m-1$

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, **by known data**, and **random**.

Initialize weights and biases with zero:

- ▶ 1st feedforward: $\mathbf{a}_1 = \mathbf{x}$, $\mathbf{z}_i = \mathbf{O}\mathbf{a}_i + \mathbf{o} = \mathbf{o}$, $\mathbf{a}_{i+1} = a_i(\mathbf{z}) = a_i(\mathbf{o})$ **constant**, $i = 1, \dots, m-1$
- ▶ 1st backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{o}) \circ (a_{m-1}(\mathbf{o}) - \mathbf{y})$, $\delta_i = a'_i(\mathbf{o}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{o}$

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, **by known data**, and **random**.

Initialize weights and biases with zero:

- ▶ 1st feedforward: $\mathbf{a}_1 = \mathbf{x}$, $\mathbf{z}_i = \mathbf{O}\mathbf{a}_i + \mathbf{o} = \mathbf{o}$, $\mathbf{a}_{i+1} = a_i(\mathbf{z}) = a_i(\mathbf{o})$ **constant**, $i = 1, \dots, m-1$
- ▶ 1st backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{o}) \circ (a_{m-1}(\mathbf{o}) - \mathbf{y})$, $\delta_i = a'_i(\mathbf{o}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{o}$
- ▶ 1st update: $\mathbf{W}_{m-1} = -\delta_{m-1} \mathbf{a}_{m-1}^\top \eta$ has all **columns equal**

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, **by known data**, and **random**.

Initialize weights and biases with zero:

- ▶ 1st feedforward: $\mathbf{a}_1 = \mathbf{x}$, $\mathbf{z}_i = \mathbf{O}\mathbf{a}_i + \mathbf{o} = \mathbf{o}$, $\mathbf{a}_{i+1} = a_i(\mathbf{z}) = a_i(\mathbf{o})$ **constant**, $i = 1, \dots, m-1$
- ▶ 1st backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{o}) \circ (a_{m-1}(\mathbf{o}) - \mathbf{y})$, $\delta_i = a'_i(\mathbf{o}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{o}$
- ▶ 1st update: $\mathbf{W}_{m-1} = -\delta_{m-1} \mathbf{a}_{m-1}^\top \eta$ has all **columns equal**
- ▶ 2nd backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{o}) \circ (a_{m-1}(\mathbf{o}) - \mathbf{y})$, $\delta_{m-2} = a'_{m-2}(\mathbf{o}) \circ \mathbf{W}_{m-1}^\top \delta_{m-1}$ is **constant**
 $\delta_i = a'_i(\mathbf{o}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{o}$

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, **by known data**, and **random**.

Initialize weights and biases with zero:

- ▶ 1st feedforward: $\mathbf{a}_1 = \mathbf{x}$, $\mathbf{z}_i = \mathbf{O}\mathbf{a}_i + \mathbf{b} = \mathbf{b}$, $\mathbf{a}_{i+1} = a_i(\mathbf{z}) = a_i(\mathbf{b})$ **constant**, $i = 1, \dots, m-1$
- ▶ 1st backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{b}) \circ (a_{m-1}(\mathbf{b}) - \mathbf{y})$, $\delta_i = a'_i(\mathbf{b}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{b}$
- ▶ 1st update: $\mathbf{W}_{m-1} = -\delta_{m-1} \mathbf{a}_{m-1}^\top \eta$ has all **columns equal**
- ▶ 2nd backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{b}) \circ (a_{m-1}(\mathbf{b}) - \mathbf{y})$, $\delta_{m-2} = a'_{m-2}(\mathbf{b}) \circ \mathbf{W}_{m-1}^\top \delta_{m-1}$ is **constant**
 $\delta_i = a'_i(\mathbf{b}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{b}$
- ▶ 2nd update: $\mathbf{W}_{m-1} = -\delta_{m-1}^{(1)} (\mathbf{a}_{m-1}^{(1)})^\top \eta^{(1)} - \delta_{m-1}^{(2)} (\mathbf{a}_{m-1}^{(2)})^\top \eta^{(2)}$ has all **columns equal**, $\mathbf{W}_{m-2}$ has all **entries equal**, $\mathbf{b}_{m-2}$ is **constant**

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, **by known data**, and **random**.

Initialize weights and biases with zero:

- ▶ 1st feedforward: $\mathbf{a}_1 = \mathbf{x}$, $\mathbf{z}_i = \mathbf{O}\mathbf{a}_i + \mathbf{o} = \mathbf{o}$, $\mathbf{a}_{i+1} = a_i(\mathbf{z}) = a_i(\mathbf{o})$ **constant**, $i = 1, \dots, m-1$
- ▶ 1st backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{o}) \circ (a_{m-1}(\mathbf{o}) - \mathbf{y})$, $\delta_i = a'_i(\mathbf{o}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{o}$
- ▶ 1st update: $\mathbf{W}_{m-1} = -\delta_{m-1} \mathbf{a}_{m-1}^\top \eta$ has all **columns equal**
- ▶ 2nd backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{o}) \circ (a_{m-1}(\mathbf{o}) - \mathbf{y})$, $\delta_{m-2} = a'_{m-2}(\mathbf{o}) \circ \mathbf{W}_{m-1}^\top \delta_{m-1}$ is **constant**
 $\delta_i = a'_i(\mathbf{o}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{o}$
- ▶ 2nd update: $\mathbf{W}_{m-1} = -\delta_{m-1}^{(1)} (\mathbf{a}_{m-1}^{(1)})^\top \eta^{(1)} - \delta_{m-1}^{(2)} (\mathbf{a}_{m-1}^{(2)})^\top \eta^{(2)}$ has all **columns equal**, $\mathbf{W}_{m-2}$ has all **entries equal**, $\mathbf{b}_{m-2}$ is **constant**
- ▶ update $m-1$: $\mathbf{W}_{m-1}$ has all **columns equal**, all other $\mathbf{W}_i$ have all **entries equal**; bias $\mathbf{b}_{m-1}$ general, other $\mathbf{b}_i$ **constant**

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, **by known data**, and **random**.

Initialize weights and biases with zero:

- ▶ 1st feedforward: $\mathbf{a}_1 = \mathbf{x}$, $\mathbf{z}_i = \mathbf{O}\mathbf{a}_i + \mathbf{o} = \mathbf{o}$, $\mathbf{a}_{i+1} = a_i(\mathbf{z}) = a_i(\mathbf{o})$ **constant**, $i = 1, \dots, m-1$
- ▶ 1st backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{o}) \circ (a_{m-1}(\mathbf{o}) - \mathbf{y})$, $\delta_i = a'_i(\mathbf{o}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{o}$
- ▶ 1st update: $\mathbf{W}_{m-1} = -\delta_{m-1} \mathbf{a}_{m-1}^\top \eta$ has all **columns equal**
- ▶ 2nd backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{o}) \circ (a_{m-1}(\mathbf{o}) - \mathbf{y})$, $\delta_{m-2} = a'_{m-2}(\mathbf{o}) \circ \mathbf{W}_{m-1}^\top \delta_{m-1}$ is **constant**
 $\delta_i = a'_i(\mathbf{o}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{o}$
- ▶ 2nd update: $\mathbf{W}_{m-1} = -\delta_{m-1}^{(1)} (\mathbf{a}_{m-1}^{(1)})^\top \eta^{(1)} - \delta_{m-1}^{(2)} (\mathbf{a}_{m-1}^{(2)})^\top \eta^{(2)}$ has all **columns equal**, $\mathbf{W}_{m-2}$ has all **entries equal**, $\mathbf{b}_{m-2}$ is **constant**
- ▶ update $m-1$: $\mathbf{W}_{m-1}$ has all **columns equal**, all other $\mathbf{W}_i$ have all **entries equal**; bias $\mathbf{b}_{m-1}$ general, other $\mathbf{b}_i$ **constant**

In case of $a_i(0) = 0$ all weight matrices and first $m-2$ biases are **always zero**.

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:

zero, by known data, and random.

↪ zero initialization does not work.

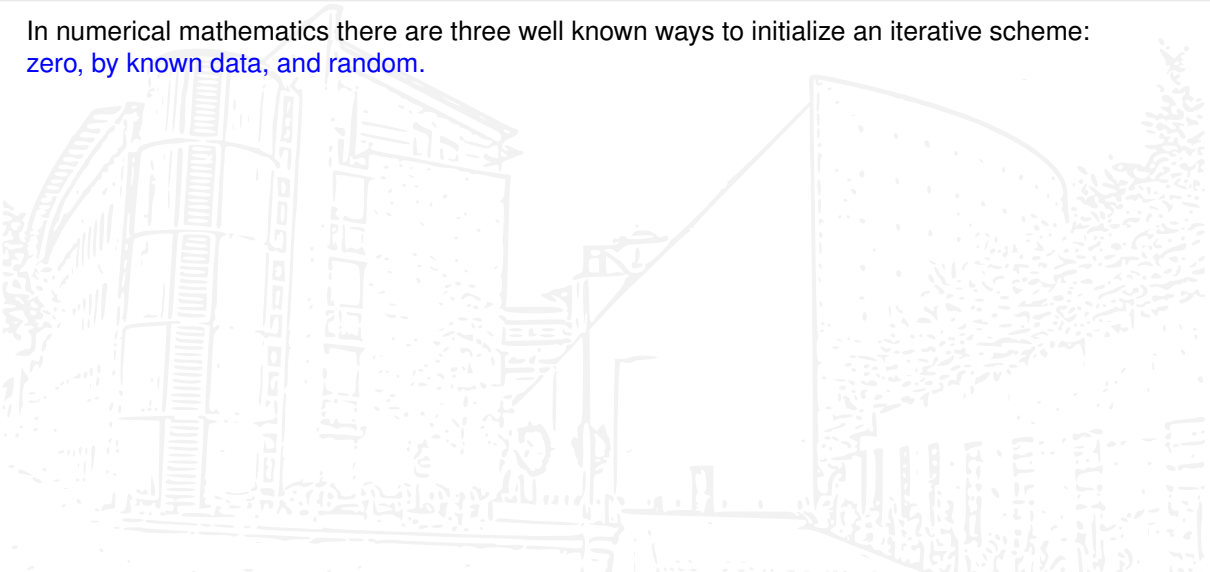
Initialize weights and biases with zero:

- ▶ 1st feedforward: $\mathbf{a}_1 = \mathbf{x}$, $\mathbf{z}_i = \mathbf{O}\mathbf{a}_i + \mathbf{o} = \mathbf{o}$, $\mathbf{a}_{i+1} = a_i(\mathbf{z}) = a_i(\mathbf{o})$ constant, $i = 1, \dots, m-1$
- ▶ 1st backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{o}) \circ (a_{m-1}(\mathbf{o}) - \mathbf{y})$, $\delta_i = a'_i(\mathbf{o}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{o}$
- ▶ 1st update: $\mathbf{W}_{m-1} = -\delta_{m-1} \mathbf{a}_{m-1}^\top \eta$ has all columns equal
- ▶ 2nd backpropagation: $\delta_{m-1} = a'_{m-1}(\mathbf{o}) \circ (a_{m-1}(\mathbf{o}) - \mathbf{y})$, $\delta_{m-2} = a'_{m-2}(\mathbf{o}) \circ \mathbf{W}_{m-1}^\top \delta_{m-1}$ is constant
 $\delta_i = a'_i(\mathbf{o}) \circ \mathbf{O}^\top \delta_{i+1} = \mathbf{o}$
- ▶ 2nd update: $\mathbf{W}_{m-1} = -\delta_{m-1}^{(1)} (\mathbf{a}_{m-1}^{(1)})^\top \eta^{(1)} - \delta_{m-1}^{(2)} (\mathbf{a}_{m-1}^{(2)})^\top \eta^{(2)}$ has all columns equal, $\mathbf{W}_{m-2}$ has all entries equal, $\mathbf{b}_{m-2}$ is constant
- ▶ update $m-1$: $\mathbf{W}_{m-1}$ has all columns equal, all other $\mathbf{W}_i$ have all entries equal; bias $\mathbf{b}_{m-1}$ general, other $\mathbf{b}_i$ constant

In case of $a_i(0) = 0$ all weight matrices and first $m-2$ biases are always zero.

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, by known data, and random.



Starting values

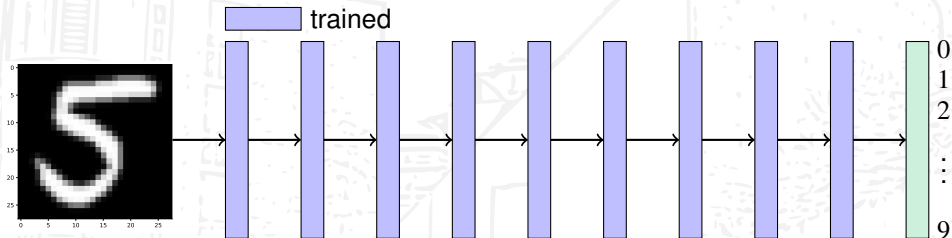
In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, by known data, and random.

Known layers? Similar problems? $\rightsquigarrow$ need past experience

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, by **known data**, and **random**.

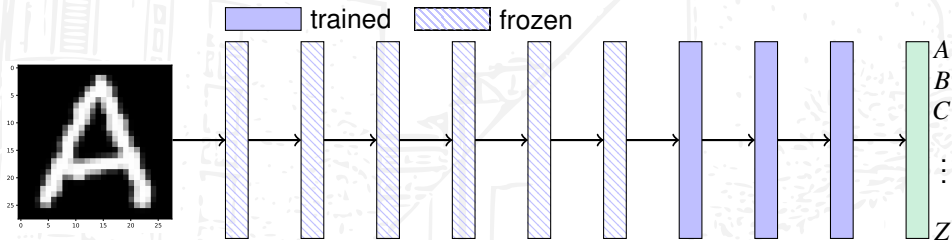
Known layers? Similar problems? $\rightsquigarrow$ need past experience



Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, by **known data**, and **random**.

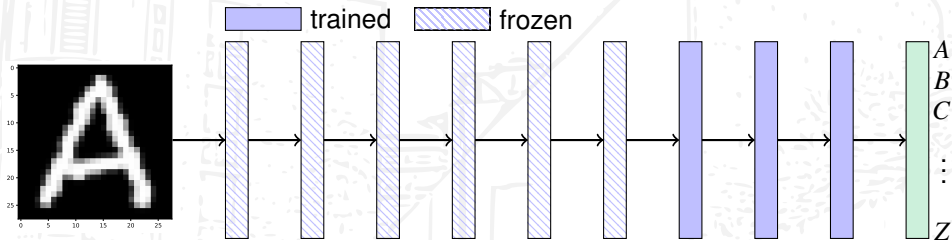
Known layers? Similar problems? $\rightsquigarrow$ need past experience



Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, **by known data**, and **random**.

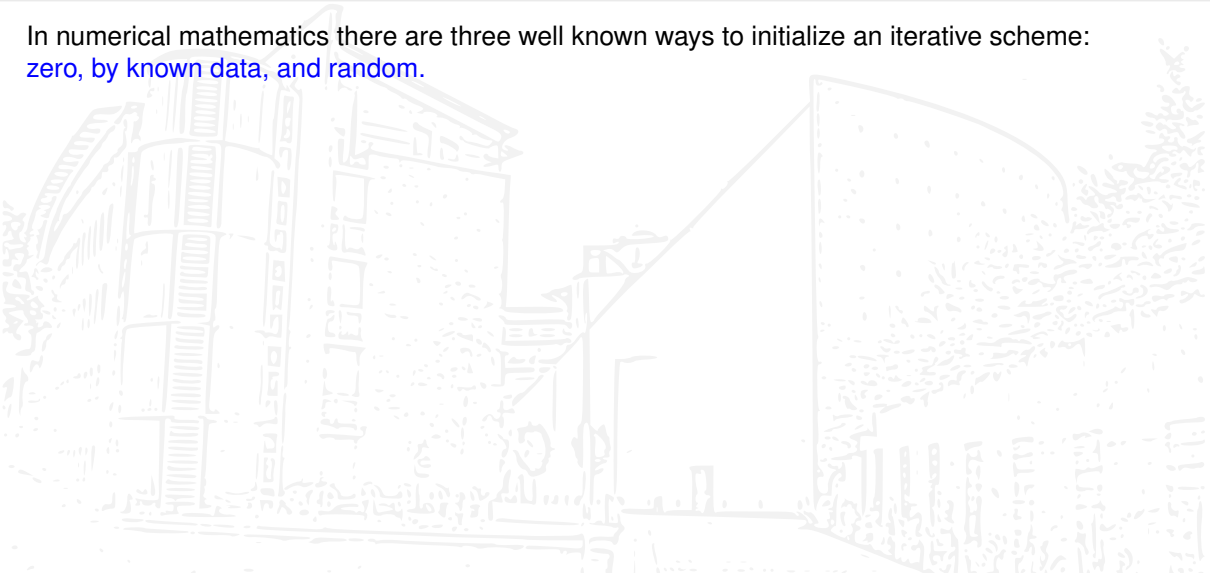
Known layers? Similar problems? $\rightsquigarrow$ need past experience



Input and output are layers: we always transform data to be of digestible input or output format, i.e., **scale and shift** to $[0, 1]$, $[-1, 1]$, or mean 0 and variance 1, depending on data and activations.

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme:
zero, by known data, and random.



Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme: zero, by known data, and random.

Initialize weights (and biases) with random values: random matrices, especially **eigenvalues** and **singular values** thereof, are well researched, see, e.g., Mehta [8] and Edelman and Rao [2].

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme: zero, by known data, and random.

Initialize weights (and biases) with random values: random matrices, especially **eigenvalues** and **singular values** thereof, are well researched, see, e.g., Mehta [8] and Edelman and Rao [2].

Let's do eigenvalues first: the **Circular Law** states that the eigenvalues of random $n \times n$ matrices with i.i.d. (independent and identically distributed) elements from distribution with zero mean $\mathbb{E}[X] = \mu$, $\mu = 0$ and variance $\mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = \sigma^2$, $\sigma^2 = 1/n$ are **uniformly distributed on the unit disc**.

Starting values

In numerical mathematics there are three well known ways to initialize an iterative scheme: zero, by known data, and random.

Initialize weights (and biases) with random values: random matrices, especially **eigenvalues** and **singular values** thereof, are well researched, see, e.g., Mehta [8] and Edelman and Rao [2].

Let's do eigenvalues first: the **Circular Law** states that the eigenvalues of random $n \times n$ matrices with i.i.d. (independent and identically distributed) elements from distribution with zero mean $\mathbb{E}[X] = \mu$, $\mu = 0$ and variance $\mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = \sigma^2$, $\sigma^2 = 1/n$ are **uniformly distributed on the unit disc**.

The distribution of singular values has been found by **Marčenko and Pastur**. Basically, the largest and smallest singular value of $\mathbf{X} \in \mathbb{R}^{n \times k}$ with $k \leq n$ and i.i.d. elements with mean zero and variance 1 behave like

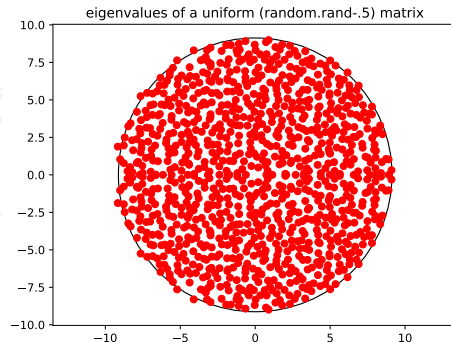
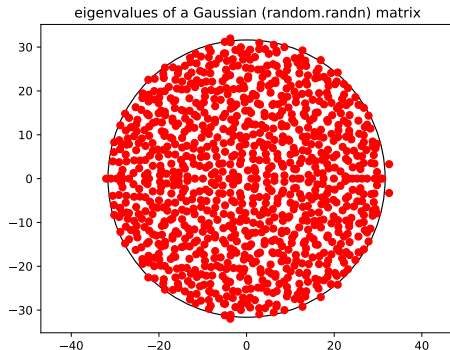
$$\left(1 \pm \frac{k}{n}\right) \sqrt{n}.$$

Starting values

By the circular law, the eigenvalues of a **Gaussian matrix** `np.random.randn(n, n)` are approximately contained in the **circle with radius $\sqrt{n}$** , those of a **shifted uniform matrix** `np.random.rand(n, n) - .5` in a **circle with radius $\sqrt{n/12}$** .

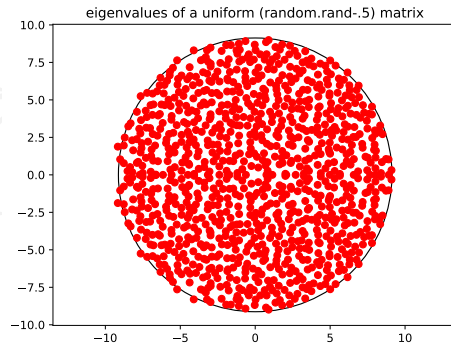
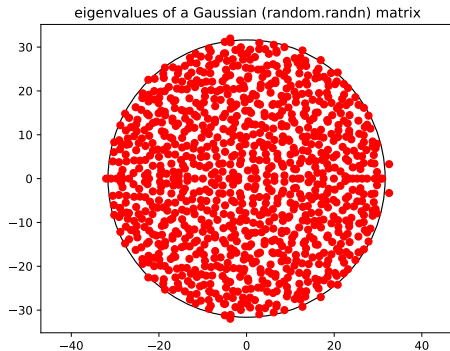
Starting values

By the circular law, the eigenvalues of a **Gaussian matrix** `np.random.randn(n, n)` are approximately contained in the **circle with radius $\sqrt{n}$** , those of a **shifted uniform matrix** `np.random.rand(n, n) - .5` in a **circle with radius $\sqrt{n/12}$** . Depicted for $n = 1000$:



Starting values

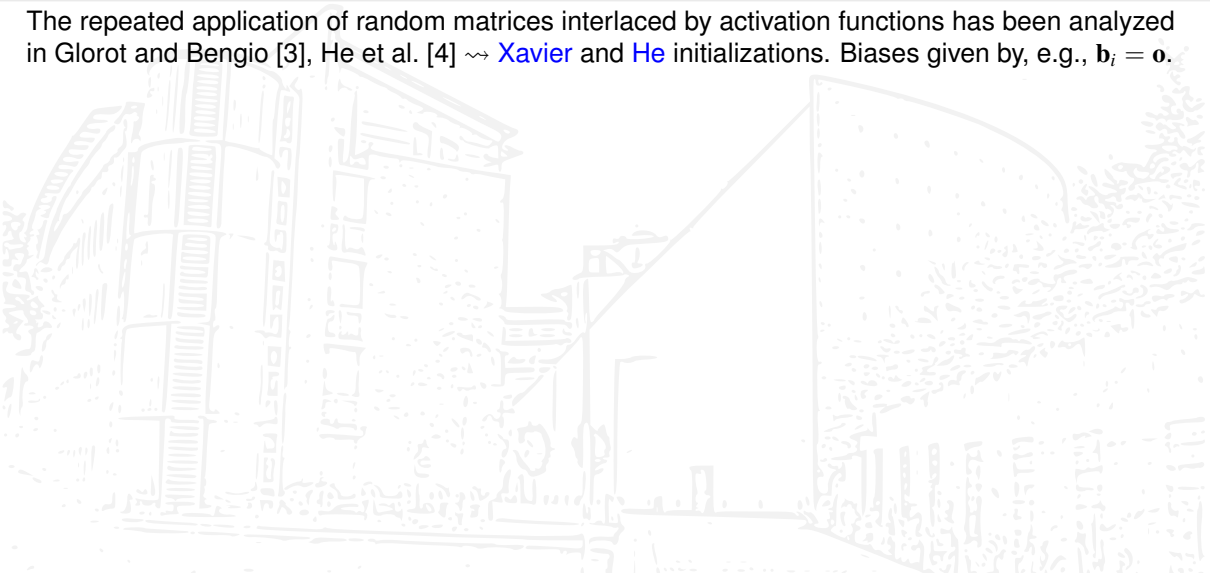
By the circular law, the eigenvalues of a **Gaussian matrix** `np.random.randn(n, n)` are approximately contained in the **circle with radius $\sqrt{n}$** , those of a **shifted uniform matrix** `np.random.rand(n, n) - .5` in a **circle with radius $\sqrt{n/12}$** . Depicted for $n = 1000$:



⇒ to ensure **good data flow** we need to select matrices w/ **zero mean** and **unit variance**

Starting values

The repeated application of random matrices interlaced by activation functions has been analyzed in Glorot and Bengio [3], He et al. [4] $\rightsquigarrow$ **Xavier** and **He** initializations. Biases given by, e.g., $\mathbf{b}_i = \mathbf{o}$.



Starting values

The repeated application of random matrices interlaced by activation functions has been analyzed in Glorot and Bengio [3], He et al. [4] $\rightsquigarrow$ **Xavier** and **He** initializations. Biases given by, e.g., $\mathbf{b}_i = \mathbf{o}$.

Random starting values: uniform distribution on $[-r, r]$, σ standard deviation.

activation function	uniform distribution	normal distribution
hyperbolic tangent	$r = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}$
ReLU	$r = \sqrt{\frac{12}{n_{\text{in}} + n_{\text{out}}}}$	$\sigma = \sqrt{\frac{4}{n_{\text{in}} + n_{\text{out}}}}$

Starting values

The repeated application of random matrices interlaced by activation functions has been analyzed in Glorot and Bengio [3], He et al. [4] $\rightsquigarrow$ **Xavier** and **He** initializations. Biases given by, e.g., $\mathbf{b}_i = \mathbf{o}$.

Random starting values: uniform distribution on $[-r, r]$, σ standard deviation.

activation function	uniform distribution	normal distribution
hyperbolic tangent	$r = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}$
ReLU	$r = \sqrt{\frac{12}{n_{\text{in}} + n_{\text{out}}}}$	$\sigma = \sqrt{\frac{4}{n_{\text{in}} + n_{\text{out}}}}$

$\times \sqrt{2}$

Starting values

The repeated application of random matrices interlaced by activation functions has been analyzed in Glorot and Bengio [3], He et al. [4] $\rightsquigarrow$ **Xavier** and **He** initializations. Biases given by, e.g., $\mathbf{b}_i = \mathbf{o}$.

Random starting values: uniform distribution on $[-r, r]$, σ standard deviation.

activation function	uniform distribution	normal distribution
hyperbolic tangent	$r = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}$
ReLU	$r = \sqrt{\frac{12}{n_{\text{in}} + n_{\text{out}}}}$	$\sigma = \sqrt{\frac{4}{n_{\text{in}} + n_{\text{out}}}}$

$\times \sqrt{2}$

- **TensorFlow:** <https://keras.io/api/layers/initializers/>
- **PyTorch:** gain factor 5/3 for TanH, <https://pytorch.org/docs/stable/nn.init.html>
- **Abs:** unknown, perhaps gain factor 1 is best

Starting values

Saxe, McClelland, and Ganguli [10] suggest to use random $m \times n$ **orthonormal matrices**.

For these, all **eigenvalues** ($m = n$) lie **on the unit circle** and all **singular values** are **one**.

Starting values

Saxe, McClelland, and Ganguli [10] suggest to use random $m \times n$ **orthonormal matrices**.

For these, all **eigenvalues** ($m = n$) lie **on the unit circle** and all **singular values** are **one**.

To compute **random** orthonormal matrices, one can use the **SVD**:

```
 $\mathbf{A} = \text{np.random.randn}(m, n),$   
 $\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}^T = \text{np.linalg.svd}(\mathbf{A}, \text{full\_matrices=False}).$ 
```

Depending on the dimensions one uses $\mathbf{W} = \mathbf{U}$ ($m \geq n$) or $\mathbf{W} = \mathbf{V}^T$ ($m \leq n$).

Starting values

Saxe, McClelland, and Ganguli [10] suggest to use random $m \times n$ **orthonormal matrices**.

For these, all **eigenvalues** ($m = n$) lie **on the unit circle** and all **singular values** are **one**.

To compute **random** orthonormal matrices, one can use the **SVD**:

```
 $\mathbf{A} = \text{np.random.randn}(m, n),$   
 $\mathbf{U}, \mathbf{\Sigma}, \mathbf{V}^T = \text{np.linalg.svd}(\mathbf{A}, \text{full\_matrices=False}).$ 
```

Depending on the dimensions one uses $\mathbf{W} = \mathbf{U}$ ($m \geq n$) or $\mathbf{W} = \mathbf{V}^T$ ($m \leq n$).

Cheaper approaches are based on the **QR decomposition** (here only for $m \geq n$)

```
 $\mathbf{Q}, \mathbf{R} = \text{np.linalg.qr}(\mathbf{A}),$   
 $\mathbf{W} = \mathbf{Q} * \text{np.sign}(\text{np.diag}(\mathbf{R})),$ 
```

or on a product of **Householder reflectors**.

Outline

Repetition

- Feedforward
- Backpropagation

Initialization

- Zero/known/random weights
- Xavier Glorot, Kaiming He, ...

Heuristic variants of (S)GD

- Momentum & NAG
- Adaptive learning rates (Adagrad, RMSprop)
- Combinations (Adam, Nadam)

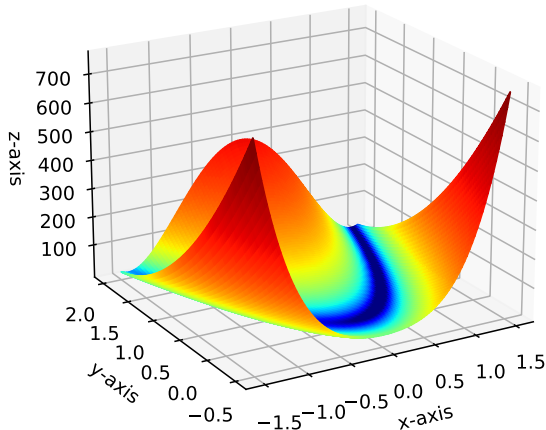
Regularization

- L^2 and L^1 regularization
- Batch normalization
- Dropout & DropConnect

Enhancements of (S)GD

(S)GD w/ minibatches **suffers from**:

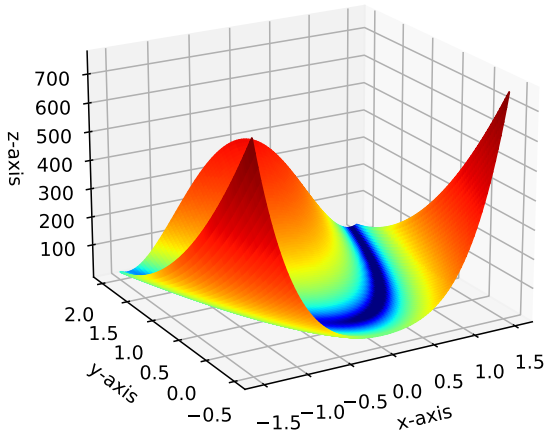
- 1) **oscillating** slow convergence in “banana-shaped valleys”, e.g., the so-called Rosenbrock function;



Enhancements of (S)GD

(S)GD w/ minibatches **suffers from**:

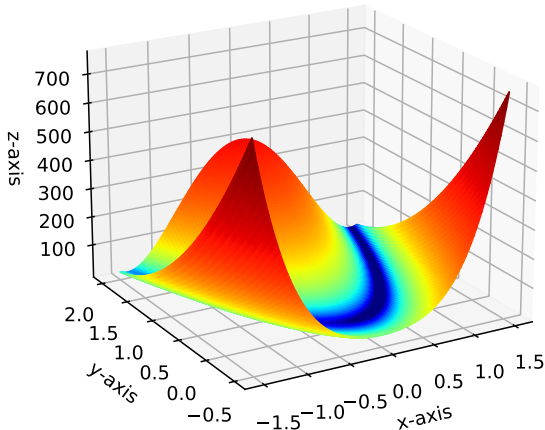
- 1) **oscillating** slow convergence in “banana-shaped valleys”, e.g., the so-called Rosenbrock function; **gets stuck** in unwanted saddle points



Enhancements of (S)GD

(S)GD w/ minibatches **suffers from**:

- 1) **oscillating** slow convergence in “banana-shaped valleys”, e.g., the so-called Rosenbrock function; **gets stuck** in unwanted saddle points
- 2) matching **learning rate** has to be found & **adapted** in the course of the computation

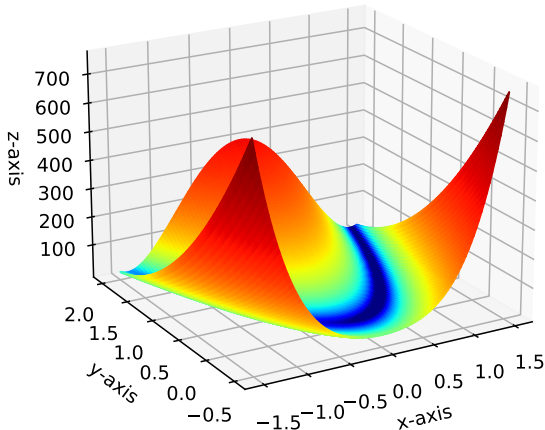


Enhancements of (S)GD

(S)GD w/ minibatches **suffers from**:

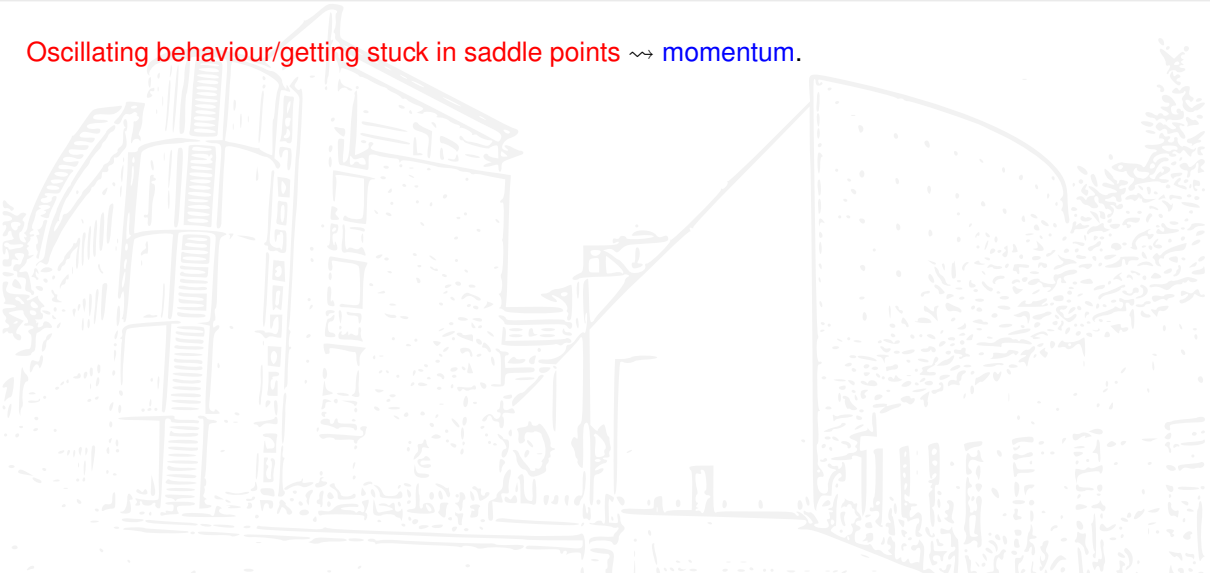
- 1) **oscillating** slow convergence in “banana-shaped valleys”, e.g., the so-called Rosenbrock function; **gets stuck** in unwanted saddle points
- 2) matching **learning rate** has to be found & **adapted** in the course of the computation

On the next slides we consider improvements, grouped into 1), 2), 1) and 2).



Speeding up with momentum

Oscillating behaviour/getting stuck in saddle points $\rightsquigarrow$ momentum.



Speeding up with momentum

Oscillating behaviour/getting stuck in saddle points $\rightsquigarrow$ momentum.

SGD update (w/ minibatches) and given learning rate η :

$$\mathbf{p} \leftarrow \mathbf{p} - \nabla C_j(\mathbf{p})\eta.$$

Speeding up with momentum

Oscillating behaviour/getting stuck in saddle points $\rightsquigarrow$ momentum.

SGD update (w/ minibatches) and given learning rate η :

$$\mathbf{p} \leftarrow \mathbf{p} - \nabla C_j(\mathbf{p})\eta.$$

Modification: auxiliary vector $\mathbf{v}$ with exponentially decaying ($\gamma < 1$, mostly $\gamma = 0.9$) combination of past updates:

$$\mathbf{v} \leftarrow \mathbf{v}\gamma + \nabla C_j(\mathbf{p})\eta, \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{v}.$$

Initialization: $\mathbf{v} = \mathbf{o}$. (Other variants exist!)

Speeding up with momentum

Oscillating behaviour/getting stuck in saddle points $\rightsquigarrow$ momentum.

SGD update (w/ minibatches) and given learning rate η :

$$\mathbf{p} \leftarrow \mathbf{p} - \nabla C_j(\mathbf{p})\eta.$$

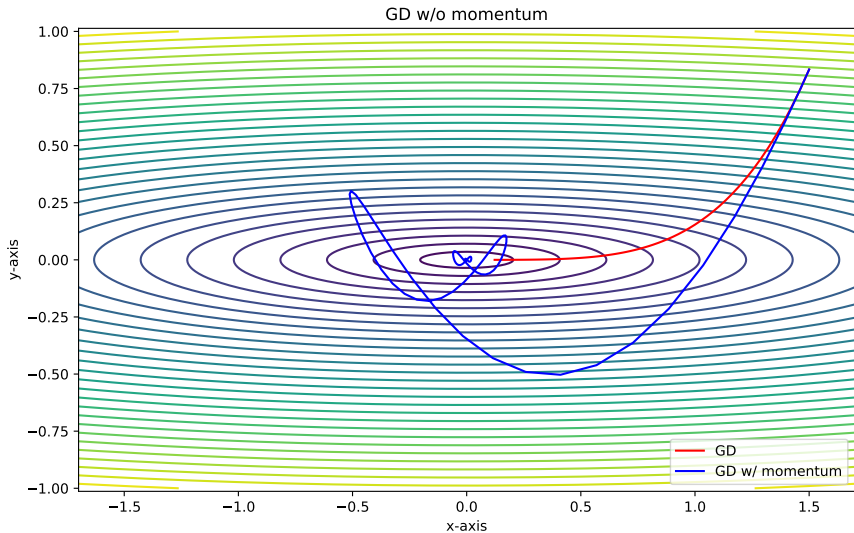
Modification: auxiliary vector $\mathbf{v}$ with exponentially decaying ($\gamma < 1$, mostly $\gamma = 0.9$) combination of past updates:

$$\mathbf{v} \leftarrow \mathbf{v}\gamma + \nabla C_j(\mathbf{p})\eta, \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{v}.$$

Initialization: $\mathbf{v} = \mathbf{o}$. (Other variants exist!)

SGD w/ momentum: rolling ball that gains momentum from past gradients until reached maximal velocity due to the friction of the air ($\gamma < 1$). Rolls past saddle points, finds faster direction of descent in steep canyons.

(S)GD w/ momentum



NAG

New method $\rightsquigarrow$ **new problems**: ball does not stop immediately at minimum, rolls upwards a bit on other side.



NAG

New method $\rightsquigarrow$ **new problems**: ball does not stop immediately at minimum, rolls upwards a bit on other side.

Nesterov: compute the derivative at an approximation of the future point,

$$\mathbf{v} \leftarrow \mathbf{v}\gamma + \nabla C_j(\mathbf{p} - \mathbf{v}\gamma)\eta, \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{v}.$$

Variant is composed of **predictor** and **corrector step**.

NAG

New method $\rightsquigarrow$ **new problems**: ball does not stop immediately at minimum, rolls upwards a bit on other side.

Nesterov: compute the derivative at an approximation of the future point,

$$\mathbf{v} \leftarrow \mathbf{v}\gamma + \nabla C_j(\mathbf{p} - \mathbf{v}\gamma)\eta, \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{v}.$$

Variant is composed of **predictor** and **corrector step**.

Nesterov's Accelerated Gradient (NAG): some kind of “forward looking ball”.

Reacts faster than (S)GD w/ momentum to changes, better choice in recurrent neural networks.

NAG

New method $\rightsquigarrow$ **new problems**: ball does not stop immediately at minimum, rolls upwards a bit on other side.

Nesterov: compute the derivative at an approximation of the future point,

$$\mathbf{v} \leftarrow \mathbf{v}\gamma + \nabla C_j(\mathbf{p} - \mathbf{v}\gamma)\eta, \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{v}.$$

Variant is composed of **predictor** and **corrector step**.

Nesterov's Accelerated Gradient (NAG): some kind of “forward looking ball”.

Reacts faster than (S)GD w/ momentum to changes, better choice in recurrent neural networks.

But: how do we evaluate the derivative at $\tilde{\mathbf{p}} := \mathbf{p} - \mathbf{v}\gamma$ using ordinary backpropagation?

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

We rewrite the recurrence:

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\tilde{\mathbf{p}}_k)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

We rewrite the recurrence:

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\tilde{\mathbf{p}}_k)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

We rewrite the recurrence:

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\tilde{\mathbf{p}}_k)\eta, \quad \tilde{\mathbf{p}}_{k+1} = \mathbf{p}_k - \mathbf{v}_k\gamma - \mathbf{v}_k.$$

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

We rewrite the recurrence:

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\tilde{\mathbf{p}}_k)\eta, \quad \tilde{\mathbf{p}}_{k+1} = \mathbf{p}_k - \mathbf{v}_k\gamma - \mathbf{v}_k.$$

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

We rewrite the recurrence:

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\tilde{\mathbf{p}}_k)\eta, \quad \tilde{\mathbf{p}}_{k+1} = \tilde{\mathbf{p}}_k - \mathbf{v}_k\gamma + \mathbf{v}_{k-1}\gamma - \mathbf{v}_k.$$

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

We rewrite the recurrence:

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\tilde{\mathbf{p}}_k)\eta, \quad \tilde{\mathbf{p}}_{k+1} = \tilde{\mathbf{p}}_k - \mathbf{v}_k\gamma + \mathbf{v}_{k-1}\gamma - \mathbf{v}_k.$$

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

We rewrite the recurrence:

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\tilde{\mathbf{p}}_k)\eta, \quad \tilde{\mathbf{p}}_{k+1} = \tilde{\mathbf{p}}_k - \mathbf{v}_k\gamma + \mathbf{v}_{k-1}\gamma - \mathbf{v}_{k-1}\gamma - \nabla C_j(\tilde{\mathbf{p}}_k)\eta.$$

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

We rewrite the recurrence:

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\tilde{\mathbf{p}}_k)\eta, \quad \tilde{\mathbf{p}}_{k+1} = \tilde{\mathbf{p}}_k - \mathbf{v}_k\gamma + \mathbf{v}_{k-1}\gamma - \mathbf{v}_{k-1}\gamma - \nabla C_j(\tilde{\mathbf{p}}_k)\eta.$$

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

We rewrite the recurrence:

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\tilde{\mathbf{p}}_k)\eta, \quad \tilde{\mathbf{p}}_{k+1} = \tilde{\mathbf{p}}_k - \mathbf{v}_k\gamma - \nabla C_j(\tilde{\mathbf{p}}_k)\eta.$$

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

We rewrite the recurrence:

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\tilde{\mathbf{p}}_k)\eta, \quad \tilde{\mathbf{p}}_{k+1} = \tilde{\mathbf{p}}_k - \mathbf{v}_k\gamma - \nabla C_j(\tilde{\mathbf{p}}_k)\eta.$$

How do we recover the original set of parameters $\mathbf{p}$?

NAG rewritten

Nesterov: original update given by

$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\mathbf{p}_k - \mathbf{v}_{k-1}\gamma)\eta, \quad \mathbf{p}_{k+1} = \mathbf{p}_k - \mathbf{v}_k.$$

How do we evaluate the derivative at $\tilde{\mathbf{p}}_k$, where

$$\tilde{\mathbf{p}}_k := \mathbf{p}_k - \mathbf{v}_{k-1}\gamma, \quad \mathbf{p}_k = \tilde{\mathbf{p}}_k + \mathbf{v}_{k-1}\gamma?$$

We rewrite the recurrence:

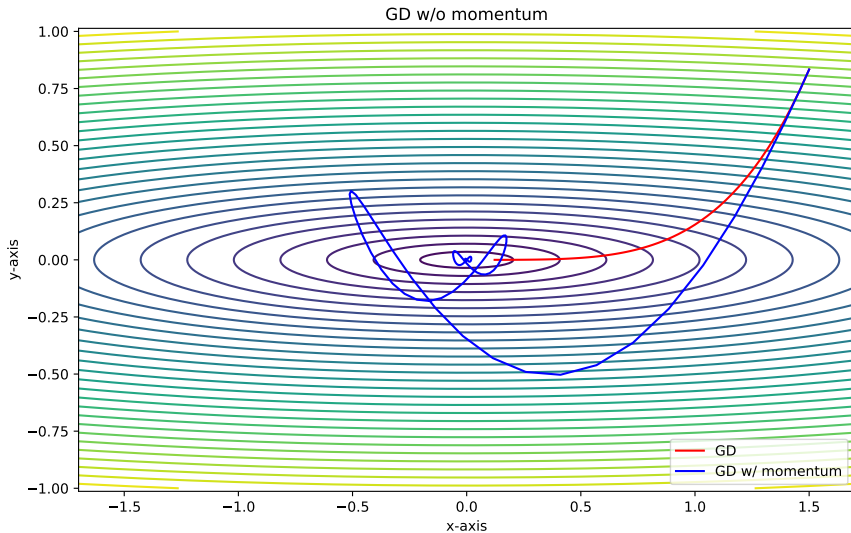
$$\mathbf{v}_k = \mathbf{v}_{k-1}\gamma + \nabla C_j(\tilde{\mathbf{p}}_k)\eta, \quad \tilde{\mathbf{p}}_{k+1} = \tilde{\mathbf{p}}_k - \mathbf{v}_k\gamma - \nabla C_j(\tilde{\mathbf{p}}_k)\eta.$$

How do we recover the original set of parameters $\mathbf{p}$?

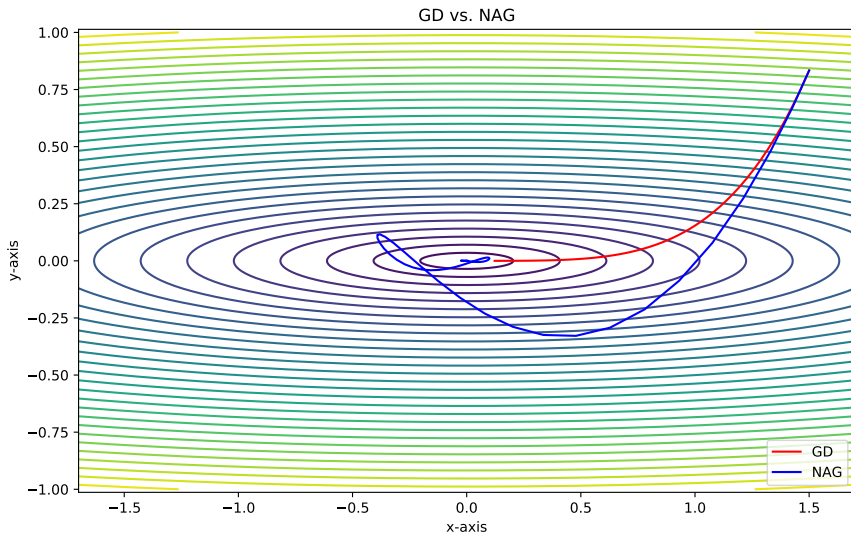
We don't. If $\mathbf{v} \rightarrow \mathbf{o}$, $\tilde{\mathbf{p}} \rightarrow \mathbf{p}$.

↪ Keras/TensorFlow & PyTorch

(S)GD w/ momentum (for comparison)



NAG



Adapting the learning rate

Experience w/ (S)GD (w/ minibatches) tells us:

- ▶ initially better to use rather large learning rates
- ▶ reduce learning rate when approaching minimum

Adapting the learning rate

Experience w/ (S)GD (w/ minibatches) tells us:

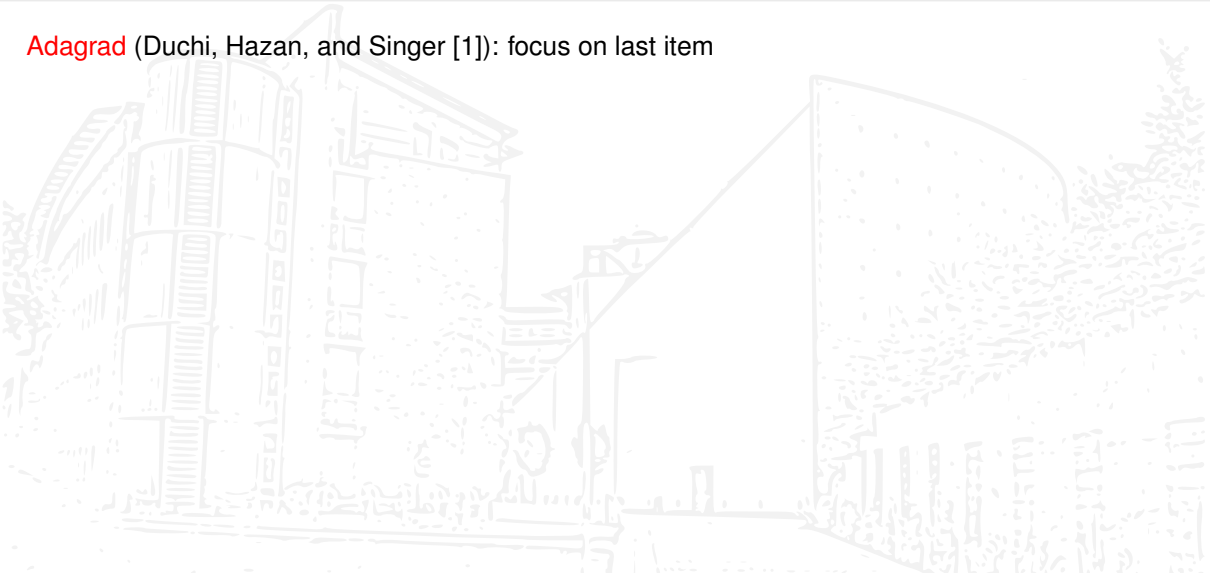
- ▶ initially better to use rather large learning rates
- ▶ reduce learning rate when approaching minimum

High dimensionality:

- ▶ fixed learning rate for fixed number of epochs $\rightsquigarrow$ hard to predict behaviour
- ▶ automatically adaptively select learning rate in the course of the training
- ▶ learning rate not equally large for all parameters, in some directions no more progress necessary

Adagrad

Adagrad (Duchi, Hazan, and Singer [1]): focus on last item



Adagrad

Adagrad (Duchi, Hazan, and Singer [1]): focus on last item

Initialize $\mathbf{w} \leftarrow \mathbf{0}$, **update**:

$$\mathbf{g} \leftarrow \nabla C_j(\mathbf{p}), \quad \mathbf{w} \leftarrow \mathbf{w} + \mathbf{g} \circ \mathbf{g}, \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{g} \circ \frac{\eta}{\sqrt{\mathbf{w} + \epsilon}},$$

where addition, square root, and quotient is interpreted **componentwise**.

Adagrad

Adagrad (Duchi, Hazan, and Singer [1]): focus on last item

Initialize $\mathbf{w} \leftarrow \mathbf{0}$, **update**:

$$\mathbf{g} \leftarrow \nabla C_j(\mathbf{p}), \quad \mathbf{w} \leftarrow \mathbf{w} + \mathbf{g} \circ \mathbf{g}, \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{g} \circ \frac{\eta}{\sqrt{\mathbf{w} + \epsilon}},$$

where addition, square root, and quotient is interpreted **componentwise**.

To **prevent division by zero** or small number choose, e.g., $\epsilon = 10^{-8}$.

Adagrad

Adagrad (Duchi, Hazan, and Singer [1]): focus on last item

Initialize $\mathbf{w} \leftarrow \mathbf{0}$, **update**:

$$\mathbf{g} \leftarrow \nabla C_j(\mathbf{p}), \quad \mathbf{w} \leftarrow \mathbf{w} + \mathbf{g} \circ \mathbf{g}, \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{g} \circ \frac{\eta}{\sqrt{\mathbf{w} + \epsilon}},$$

where addition, square root, and quotient is interpreted **componentwise**.

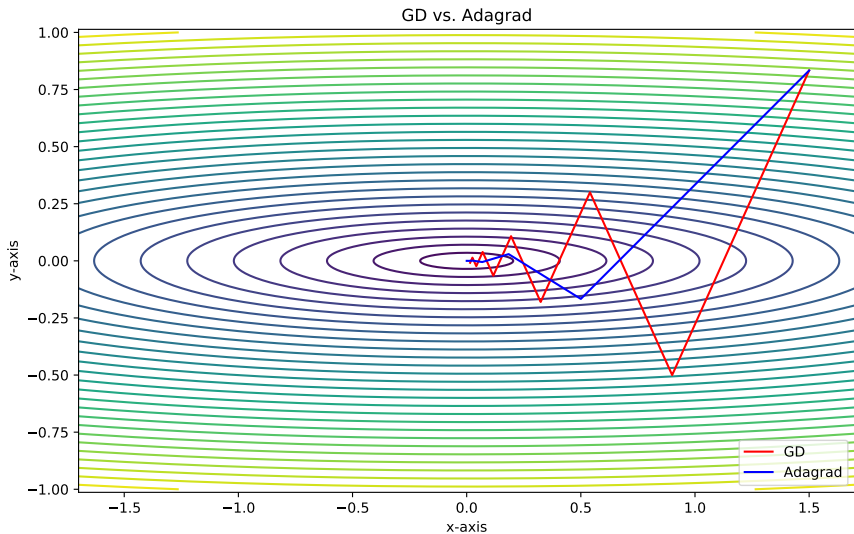
To **prevent division by zero** or small number choose, e.g., $\epsilon = 10^{-8}$.

Elements of $\mathbf{w}$ **increase monotonically**,

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \nabla C_j(\mathbf{p}) \circ \nabla C_j(\mathbf{p}) \geq \mathbf{w}_k.$$

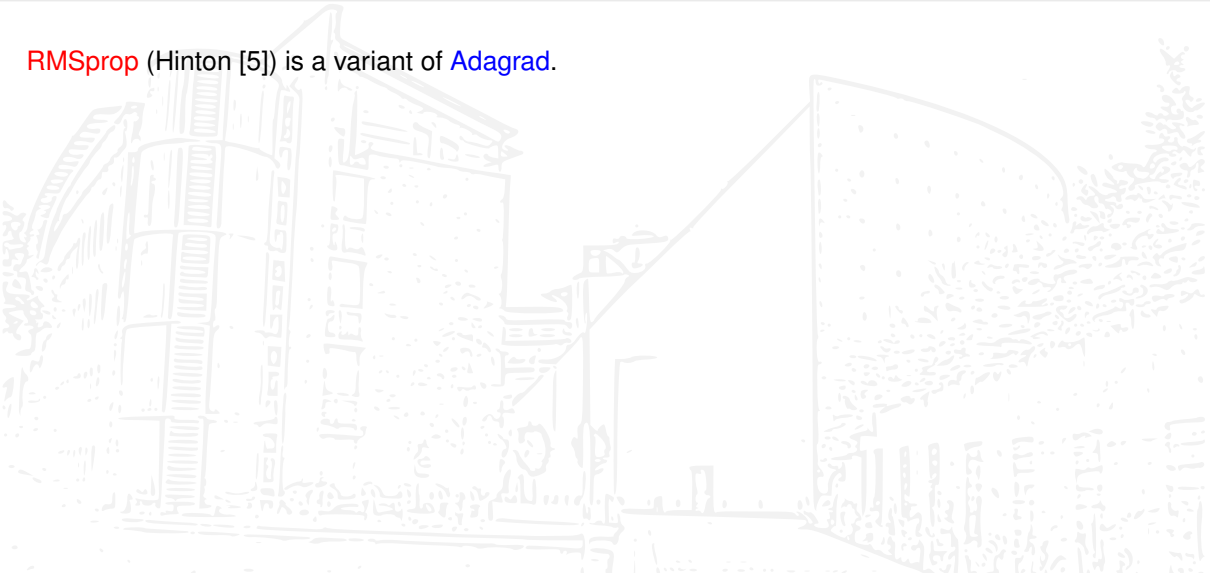
Step length can converge to zero, Adagrad gets stuck.

Adagrad



RMSprop

RMSprop (Hinton [5]) is a variant of **Adagrad**.



RMSprop

RMSprop (Hinton [5]) is a variant of **Adagrad**.

Similar to **(S)GD with momentum**, RMSprop uses **decay parameter** $\gamma < 1$ and **vector** $\mathbf{w}$ that stores decaying average of componentwise squares,

$$\mathbf{g} \leftarrow \nabla C_j(\mathbf{p}), \quad \mathbf{w} \leftarrow \mathbf{w}\gamma + \mathbf{g}^2(1 - \gamma), \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{g} \circ \frac{\eta}{\sqrt{\mathbf{w} + \epsilon}}.$$

RMSprop

RMSprop (Hinton [5]) is a variant of **Adagrad**.

Similar to **(S)GD with momentum**, RMSprop uses **decay parameter** $\gamma < 1$ and **vector** $\mathbf{w}$ that stores decaying average of componentwise squares,

$$\mathbf{g} \leftarrow \nabla C_j(\mathbf{p}), \quad \mathbf{w} \leftarrow \mathbf{w}\gamma + \mathbf{g}^2(1 - \gamma), \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{g} \circ \frac{\eta}{\sqrt{\mathbf{w} + \epsilon}}.$$

Here, square ($\mathbf{g}^2$), square root ($\sqrt{\mathbf{w}}$), and division are to be understood **componentwise**, $\circ$ denotes the **componentwise** (Schur/Hadamard) product.

RMSprop

RMSprop (Hinton [5]) is a variant of **Adagrad**.

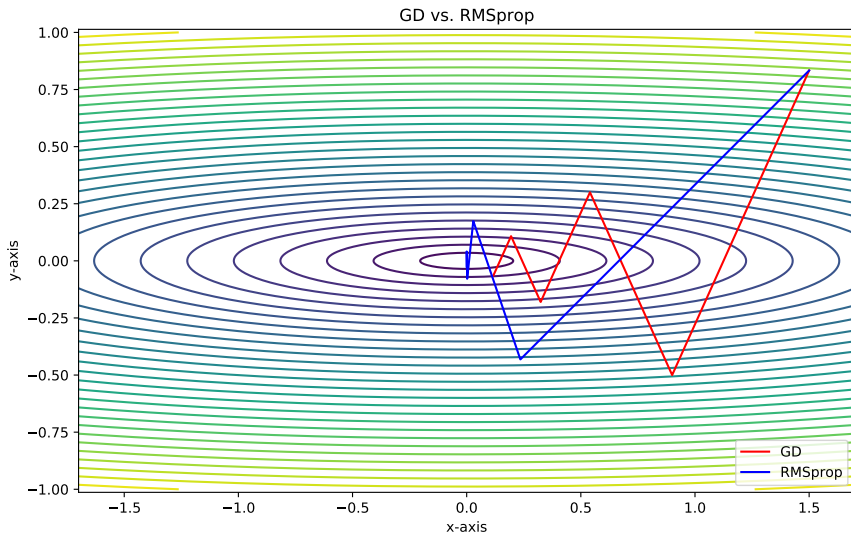
Similar to **(S)GD with momentum**, RMSprop uses **decay parameter** $\gamma < 1$ and **vector** $\mathbf{w}$ that stores decaying average of componentwise squares,

$$\mathbf{g} \leftarrow \nabla C_j(\mathbf{p}), \quad \mathbf{w} \leftarrow \mathbf{w}\gamma + \mathbf{g}^2(1 - \gamma), \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{g} \circ \frac{\eta}{\sqrt{\mathbf{w} + \epsilon}}.$$

Here, square ($\mathbf{g}^2$), square root ($\sqrt{\mathbf{w}}$), and division are to be understood **componentwise**, $\circ$ denotes the **componentwise** (Schur/Hadamard) product.

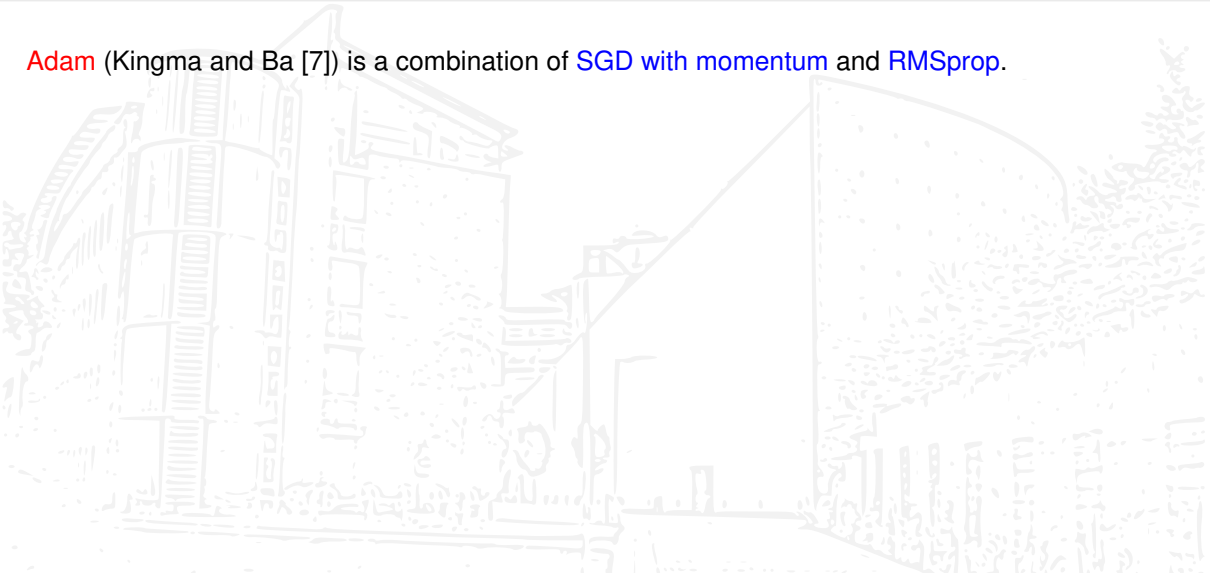
Another variant is **Adadelta** (Zeiler [13]). Adadelta even works without a given learning rate but is pretty **similar to RMSprop**, thus we omit Adadelta.

RMSprop



Adam

Adam (Kingma and Ba [7]) is a combination of **SGD with momentum** and **RMSprop**.



Adam

Adam (Kingma and Ba [7]) is a combination of **SGD with momentum** and **RMSprop**.

Adam is based on decay parameters $\beta_1 < \beta_2 < 1$ and decaying averages for the first and second moments, i.e., the gradient and the componentwise square,

$$\mathbf{g} \leftarrow \nabla C_j(\mathbf{p}), \quad \mathbf{v} \leftarrow \mathbf{v}\beta_1 + \mathbf{g}(1 - \beta_1), \quad \mathbf{w} \leftarrow \mathbf{w}\beta_2 + \mathbf{g}^2(1 - \beta_2), \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{v} \circ \frac{\eta}{\sqrt{\mathbf{w} + \epsilon}}.$$

The vectors are initialized by zero, $\mathbf{v} = \mathbf{w} = \mathbf{0}$.

Adam

Adam (Kingma and Ba [7]) is a combination of **SGD with momentum** and **RMSprop**.

Adam is based on decay parameters $\beta_1 < \beta_2 < 1$ and decaying averages for the first and second moments, i.e., the gradient and the componentwise square,

$$\mathbf{g} \leftarrow \nabla C_j(\mathbf{p}), \quad \mathbf{v} \leftarrow \mathbf{v}\beta_1 + \mathbf{g}(1 - \beta_1), \quad \mathbf{w} \leftarrow \mathbf{w}\beta_2 + \mathbf{g}^2(1 - \beta_2), \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{v} \circ \frac{\eta}{\sqrt{\mathbf{w} + \epsilon}}.$$

The vectors are initialized by zero, $\mathbf{v} = \mathbf{w} = \mathbf{0}$.

Initially, the averaged vectors have a **bias** towards zero. As a **correction** in step $k \in \mathbb{N}$ the scaling

$$\hat{\mathbf{v}} \leftarrow \mathbf{v}/(1 - \beta_1^k), \quad \hat{\mathbf{w}} \leftarrow \mathbf{w}/(1 - \beta_2^k), \quad \mathbf{p} \leftarrow \mathbf{p} - \hat{\mathbf{v}} \circ \frac{\eta}{\sqrt{\hat{\mathbf{w}} + \epsilon}}$$

is used.

Adam

An **efficient implementation** is based on a different order of computation,

$$\alpha \leftarrow \eta \cdot \frac{\sqrt{1 - \beta_2^k}}{1 - \beta_1^k}, \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{v} \circ \frac{\alpha}{\sqrt{\mathbf{w} + \hat{\epsilon}}},$$

where

$$\hat{\epsilon} = \epsilon \cdot (1 - \beta_2^k) \rightarrow \epsilon, \quad k \rightarrow \infty.$$

Typically a fixed $\hat{\epsilon}$ is used.

Adam

An **efficient implementation** is based on a different order of computation,

$$\alpha \leftarrow \eta \cdot \frac{\sqrt{1 - \beta_2^k}}{1 - \beta_1^k}, \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{v} \odot \frac{\alpha}{\sqrt{\mathbf{w} + \hat{\epsilon}}},$$

where

$$\hat{\epsilon} = \epsilon \cdot (1 - \beta_2^k) \rightarrow \epsilon, \quad k \rightarrow \infty.$$

Typically a fixed $\hat{\epsilon}$ is used.

The authors suggest the **default parameters** $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$.

Adam

An **efficient implementation** is based on a different order of computation,

$$\alpha \leftarrow \eta \cdot \frac{\sqrt{1 - \beta_2^k}}{1 - \beta_1^k}, \quad \mathbf{p} \leftarrow \mathbf{p} - \mathbf{v} \circ \frac{\alpha}{\sqrt{\mathbf{w} + \hat{\epsilon}}},$$

where

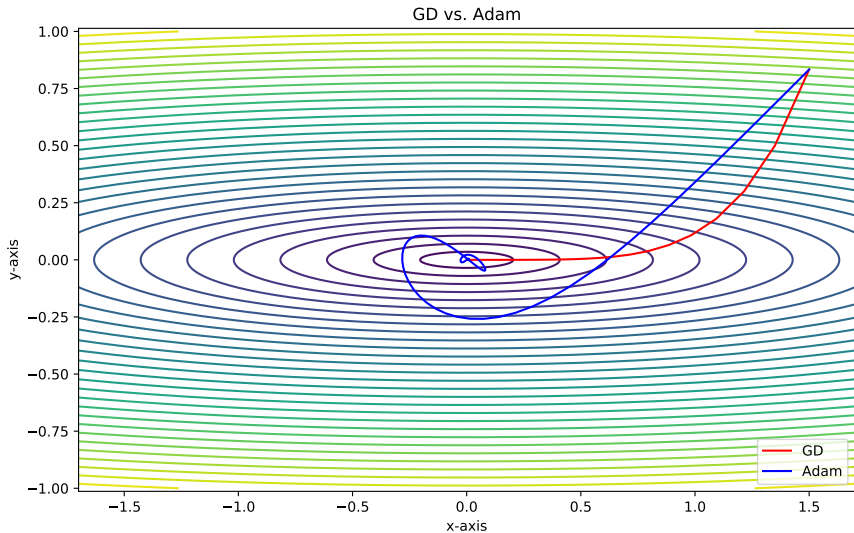
$$\hat{\epsilon} = \epsilon \cdot (1 - \beta_2^k) \rightarrow \epsilon, \quad k \rightarrow \infty.$$

Typically a fixed $\hat{\epsilon}$ is used.

The authors suggest the **default parameters** $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$.

Currently, Adam is the **standard optimizer in TensorFlow**, even though there are known problems, see Reddi, Kale, and Kumar [9].

Adam



Adamax/Nadam & beyond

Other variants are

Adamax: simplified variant of Adam based on maximum norm in place of Euclidean

Nadam: which is based on NAG in place of GD w/ momentum

Methods that do not suffer from the problems that Adam has are

AMSGrad: based on averaging first moments and taking the maximal second moment

AdamNc: based on non-constant β_1 and β_2

Outline

Repetition

- Feedforward
- Backpropagation

Initialization

- Zero/known/random weights
- Xavier Glorot, Kaiming He, ...

Heuristic variants of (S)GD

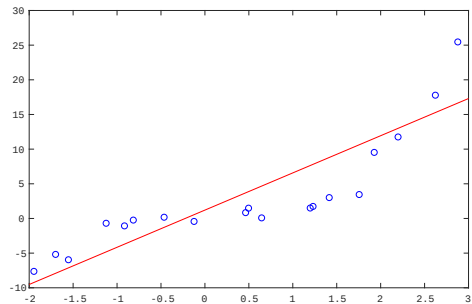
- Momentum & NAG
- Adaptive learning rates (Adagrad, RMSprop)
- Combinations (Adam, Nadam)

Regularization

- L^2 and L^1 regularization
- Batch normalization
- Dropout & DropConnect

Suitable networks vs. numerical mathematics

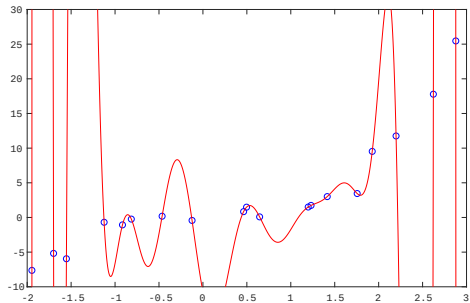
Too few neurons: **underfitting**



Suitable networks vs. numerical mathematics

Too few neurons: **underfitting**

Net learns entire data set: **overfitting**

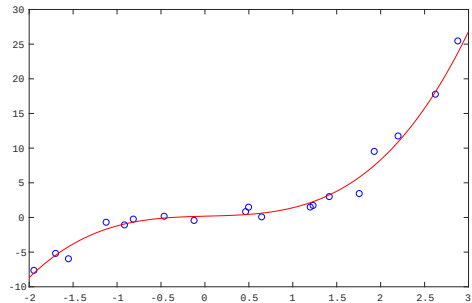


Suitable networks vs. numerical mathematics

Too few neurons: **underfitting**

Net learns entire data set: **overfitting**

Goal: **generalize** observed behaviour



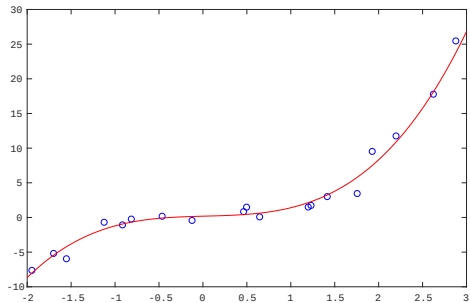
Suitable networks vs. numerical mathematics

Too few neurons: **underfitting**

Net learns entire data set: **overfitting**

Goal: **generalize** observed behaviour

Capacity: degrees of freedom including structure and activation functions



Suitable networks vs. numerical mathematics

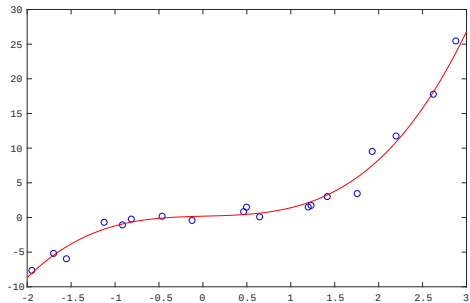
Too few neurons: **underfitting**

Net learns entire data set: **overfitting**

Goal: **generalize** observed behaviour

Capacity: degrees of freedom including structure and activation functions

Regularization: counter measures for high capacity



Suitable networks vs. numerical mathematics

Too few neurons: **underfitting**

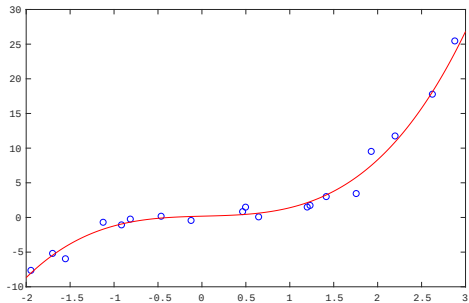
Net learns entire data set: **overfitting**

Goal: **generalize** observed behaviour

Capacity: degrees of freedom including structure and activation functions

Regularization: counter measures for high capacity

Good news: **overparameterized** NN trained w/ SGD **generalize well**



“Double Descent” phenomenon

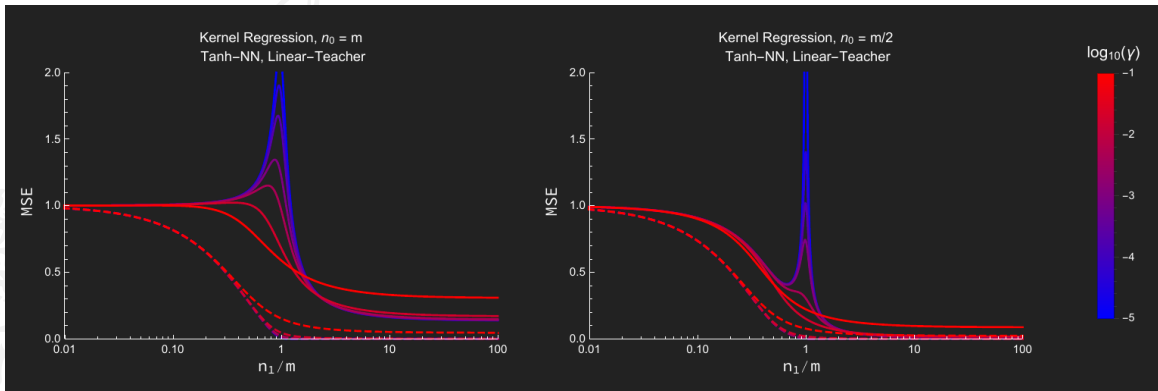
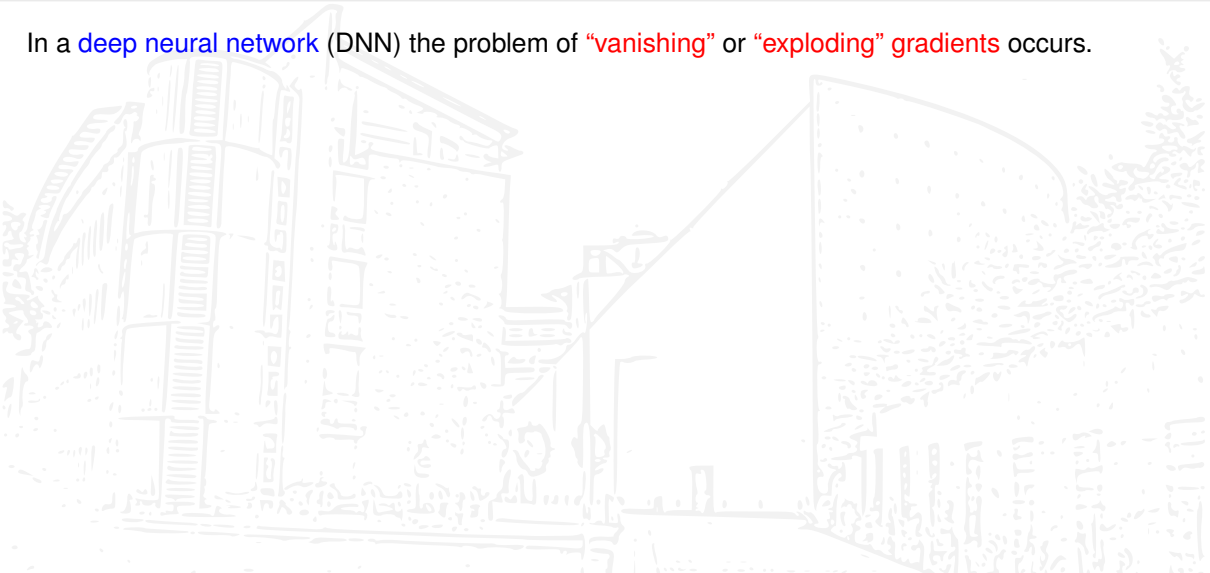


Figure taken from a talk by Jeffrey Pennington (Google Brain), 2020,
<https://cmsa.fas.harvard.edu/wp-content/uploads/2020/10/Triple-descent-and-a-fine-grained-bias-variance-decomposition.pdf>

- ▶ m : dataset size, n_0 : input dimensionality, n_1 : hidden layer size, γ : regularization parameter
- ▶ dashed: training error, lines: test error, $n_1/m = 1$: “interpolation threshold”

Exploding/vanishing gradients

In a **deep neural network** (DNN) the problem of “**vanishing**” or “**exploding**” gradients occurs.



Exploding/vanishing gradients

In a **deep neural network** (DNN) the problem of “**vanishing**” or “**exploding**” gradients occurs.

The reason lies within **backpropagation**: either the derivative of the activation function is **too small** or the entries of the weight matrices are **too large**.

Exploding/vanishing gradients

In a **deep neural network** (DNN) the problem of “**vanishing**” or “**exploding**” gradients occurs.

The reason lies within **backpropagation**: either the derivative of the activation function is **too small** or the entries of the weight matrices are **too large**.

Example: the **logistic function** has the symmetric derivative $\text{Logistic}' = \text{Logistic}(1 - \text{Logistic})$ with maximum $1/4$ at zero. In case of 100 layers, neglecting weight matrices, the gradients are scaled by

$$\frac{1}{4^{100}} \approx 6.223 \cdot 10^{-61},$$

i.e., roughly 10^{60} times smaller.

Exploding/vanishing gradients

In a **deep neural network** (DNN) the problem of “**vanishing**” or “**exploding**” gradients occurs.

The reason lies within **backpropagation**: either the derivative of the activation function is **too small** or the entries of the weight matrices are **too large**.

Example: the **logistic function** has the symmetric derivative $\text{Logistic}' = \text{Logistic}(1 - \text{Logistic})$ with maximum $1/4$ at zero. In case of 100 layers, neglecting weight matrices, the gradients are scaled by

$$\frac{1}{4^{100}} \approx 6.223 \cdot 10^{-61},$$

i.e., roughly 10^{60} times smaller.

For this reason nowadays **ReLU** is used which has a derivative of size 1 for half the real axis.

Exploding/vanishing gradients

In a **deep neural network** (DNN) the problem of “**vanishing**” or “**exploding**” gradients occurs.

The reason lies within **backpropagation**: either the derivative of the activation function is **too small** or the entries of the weight matrices are **too large**.

Example: the **logistic function** has the symmetric derivative $\text{Logistic}' = \text{Logistic}(1 - \text{Logistic})$ with maximum $1/4$ at zero. In case of 100 layers, neglecting weight matrices, the gradients are scaled by

$$\frac{1}{4^{100}} \approx 6.223 \cdot 10^{-61},$$

i.e., roughly 10^{60} times smaller.

For this reason nowadays **ReLU** is used which has a derivative of size 1 for half the real axis.

The only remaining problem is that of “**dying**” neurons: the bias is so negative that the neuron never fires. $\rightsquigarrow$ “**leaky**” **ReLU**, **Abs**

Gradient clipping & L^2 regularization

The **increasing of the weights** until repeated multiplication with weights $\|\mathbf{W}_i\|_2 \gg 1$ results in NaN can be circumvented by so-called “**gradient clipping**”, e.g.:

```
delta.clip(min=-2, max=2).
```

This is also used in the **vanishing gradient problem**, where the range is adapted to the region where the activation function **does not saturate**.

Gradient clipping & L^2 regularization

The **increasing of the weights** until repeated multiplication with weights $\|\mathbf{W}_i\|_2 \gg 1$ results in NaN can be circumvented by so-called “**gradient clipping**”, e.g.:

```
delta.clip(min=-2, max=2).
```

This is also used in the **vanishing gradient problem**, where the range is adapted to the region where the activation function **does not saturate**.

Another approach is equivalent to **classical Tikhonov regularization**. Classical Tikhonov regularization replaces the solution of an almost rank deficient linear system $\mathbf{Ax} = \mathbf{b}$,

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2 \rightsquigarrow \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \lambda \|\mathbf{x}\|_2^2$$

for carefully selected $\lambda > 0$, e.g., based on the so-called **L-curve**.

L^2 regularization

For **neural networks**, L^2 or Tikhonov regularization changes the cost function by adding a **penalty term** for the size of the weights,

$$C^{(\lambda)}(\mathbf{p}) := C(\mathbf{p}) + \frac{\lambda}{2} \sum_{\ell=1}^{m-1} \|\mathbf{W}_{\ell}\|_F^2, \quad C_j^{(\lambda)}(\mathbf{p}) := C_j(\mathbf{p}) + \frac{\lambda}{2} \sum_{\ell=1}^{m-1} \|\mathbf{W}_{\ell}\|_F^2.$$

L^2 regularization

For **neural networks**, L^2 or Tikhonov regularization changes the cost function by adding a **penalty term** for the size of the weights,

$$C^{(\lambda)}(\mathbf{p}) := C(\mathbf{p}) + \frac{\lambda}{2} \sum_{\ell=1}^{m-1} \|\mathbf{W}_{\ell}\|_F^2, \quad C_j^{(\lambda)}(\mathbf{p}) := C_j(\mathbf{p}) + \frac{\lambda}{2} \sum_{\ell=1}^{m-1} \|\mathbf{W}_{\ell}\|_F^2.$$

The only change in **backpropagation** lies in the **derivative with respect to the weights**,

$$\frac{\partial C^{(\lambda)}}{\partial \mathbf{W}_i} = \frac{\partial C}{\partial \mathbf{W}_i} + \mathbf{W}_i \lambda$$

the **update** with learning rate η changes to

$$\mathbf{W}_i \leftarrow \mathbf{W}_i - \frac{\partial C^{(\lambda)}}{\partial \mathbf{W}_i} \eta = \mathbf{W}_i (1 - \lambda \eta) - \frac{\partial C}{\partial \mathbf{W}_i} \eta.$$

L^2 regularization

For **neural networks**, L^2 or Tikhonov regularization changes the cost function by adding a **penalty term** for the size of the weights,

$$C^{(\lambda)}(\mathbf{p}) := C(\mathbf{p}) + \frac{\lambda}{2} \sum_{\ell=1}^{m-1} \|\mathbf{W}_{\ell}\|_F^2, \quad C_j^{(\lambda)}(\mathbf{p}) := C_j(\mathbf{p}) + \frac{\lambda}{2} \sum_{\ell=1}^{m-1} \|\mathbf{W}_{\ell}\|_F^2.$$

The only change in **backpropagation** lies in the **derivative with respect to the weights**,

$$\frac{\partial C^{(\lambda)}}{\partial \mathbf{W}_i} = \frac{\partial C}{\partial \mathbf{W}_i} + \mathbf{W}_i \lambda$$

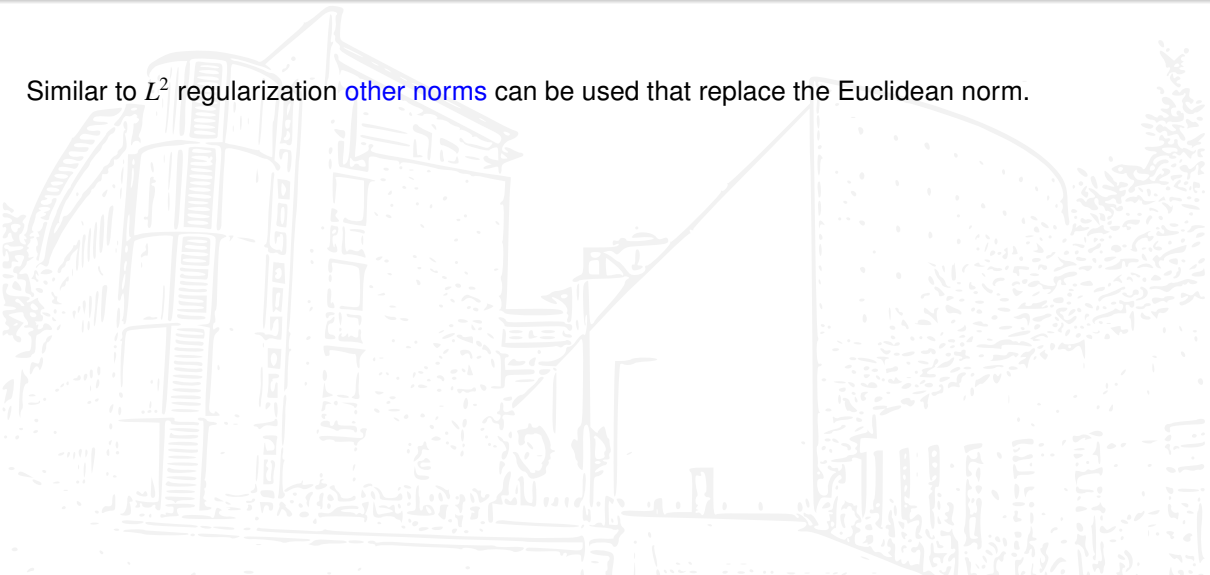
the **update** with learning rate η changes to

$$\mathbf{W}_i \leftarrow \mathbf{W}_i - \frac{\partial C^{(\lambda)}}{\partial \mathbf{W}_i} \eta = \mathbf{W}_i (1 - \lambda \eta) - \frac{\partial C}{\partial \mathbf{W}_i} \eta.$$

Some authors/packages use $\hat{\lambda} := 1 - \lambda \eta < 1$ and speak of “weight decay”. Care has to be taken!

L^1 regularization

Similar to L^2 regularization **other norms** can be used that replace the Euclidean norm.



L^1 regularization

Similar to L^2 regularization **other norms** can be used that replace the Euclidean norm.

There exists **L^1 regularization**, where in place of the Frobenius norm the **sum of absolute values of all elements** of all weight matrices is used.

L^1 regularization

Similar to L^2 regularization **other norms** can be used that replace the Euclidean norm.

There exists **L^1 regularization**, where in place of the Frobenius norm the **sum of absolute values of all elements** of all weight matrices is used.

As **replacement** for the **derivative** the sign function is used.

L^1 regularization

Similar to L^2 regularization **other norms** can be used that replace the Euclidean norm.

There exists **L^1 regularization**, where in place of the Frobenius norm the **sum of absolute values of all elements** of all weight matrices is used.

As **replacement** for the **derivative** the sign function is used.

The update of the weights becomes

$$\mathbf{W}_i \leftarrow \mathbf{W}_i - \lambda \eta \text{sign}(\mathbf{W}_i) - \frac{\partial \mathcal{C}}{\partial \mathbf{W}_i} \eta,$$

positive weights are decreased and negative ones increased by the second term in the update.

Minibatch normalization

Related to the aforementioned type of regularization is “**batch normalization**” where each element of the input to every layer is

- ▶ normalized to have mean $\mu = 0$ and variance $\sigma = 1$
- ▶ transformed linearly, such that it has mean β and variance γ

Minibatch normalization

Related to the aforementioned type of regularization is “**batch normalization**” where each element of the input to every layer is

- ▶ normalized to have mean $\mu = 0$ and variance $\sigma = 1$
- ▶ transformed linearly, such that it has mean β and variance γ

These two new parameters are not supplied by the user, they are **learned during backpropagation**.

Minibatch normalization

Related to the aforementioned type of regularization is “batch normalization” where each element of the input to every layer is

- ▶ normalized to have mean $\mu = 0$ and variance $\sigma = 1$
- ▶ transformed linearly, such that it has mean β and variance γ

These two new parameters are not supplied by the user, they are learned during backpropagation. Computing empirical mean and variance of a batch of size m is a differentiable transformation,

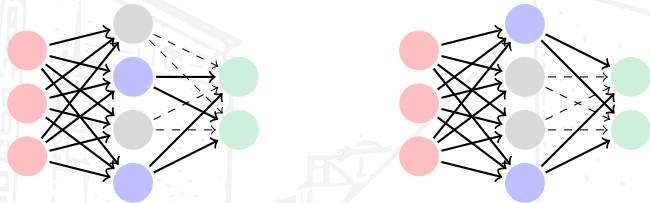
$$\mu_B := \frac{1}{m} \sum_{j=1}^m \mathbf{x}_j, \quad \sigma_B := \frac{1}{m} \sum_{j=1}^m (\mathbf{x}_j - \mu_B)^2,$$

normalization and a linear transformation too,

$$\hat{\mathbf{x}}_j = \frac{\mathbf{x}_j - \mu_B}{\sqrt{\sigma_B + \epsilon}}, \quad \mathbf{y}_j = \gamma \hat{\mathbf{x}}_j + \beta.$$

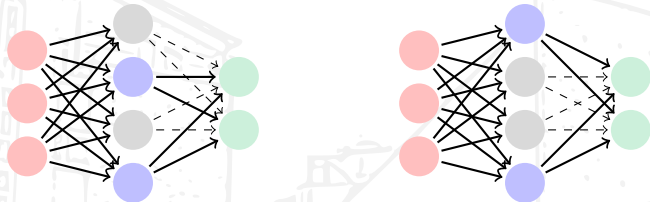
Dropout

The idea behind **Dropout** (Hinton et al. [6] and Srivastava et al. [11]) is completely different. In Dropout **neurons are kept with probability $p \in [0, 1]$** during training. Typically, $p \in [0.5, 0.8]$.



Dropout

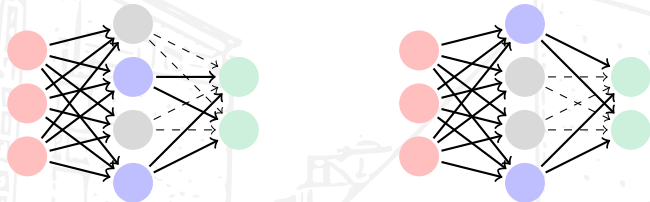
The idea behind **Dropout** (Hinton et al. [6] and Srivastava et al. [11]) is completely different. In Dropout **neurons are kept with probability $p \in [0, 1]$** during training. Typically, $p \in [0.5, 0.8]$.



After training all weights in that layer are multiplied by p . The authors argue that for $p = 0.5$ this corresponds to the **simultaneous training of 2^n nets** (for n neurons in the layer) and averaging afterwards. This can be implemented using the true weights all the time.

Dropout

The idea behind **Dropout** (Hinton et al. [6] and Srivastava et al. [11]) is completely different. In Dropout **neurons are kept with probability $p \in [0, 1]$** during training. Typically, $p \in [0.5, 0.8]$.



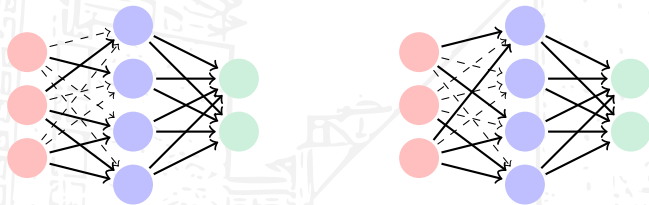
After training all weights in that layer are multiplied by p . The authors argue that for $p = 0.5$ this corresponds to the **simultaneous training of 2^n nets** (for n neurons in the layer) and averaging afterwards. This can be implemented using the true weights all the time.

Dropout can be implemented using **mask vectors $\mathbf{m} \in \{0, 1\}^n$** ,

$$\mathbf{a}_2 \leftarrow \mathbf{m} \odot \mathbf{a}_2, \quad \mathbf{m} = \text{np.random.binomial}(1, p, \text{size}=(n, 1))$$

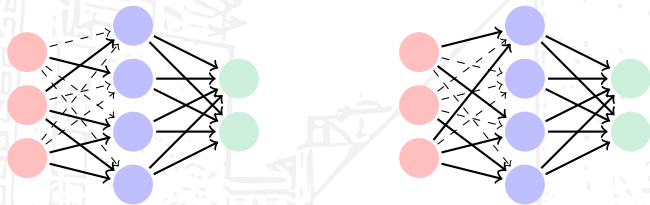
DropConnect

In **DropConnect** (Wan et al. [12]) **connections are dropped randomly**. DropConnect is a generalization of **Dropout**, Dropout is recovered by dropping all connections in DropConnect corresponding to the neuron dropped out in Dropout.



DropConnect

In **DropConnect** (Wan et al. [12]) **connections are dropped randomly**. DropConnect is a generalization of **Dropout**, Dropout is recovered by dropping all connections in DropConnect corresponding to the neuron dropped out in Dropout.



DropConnect is implemented using mask matrices $\mathbf{M} \in \{0, 1\}^{n_2 \times n_1}$,

$$\mathbf{W}_1 \leftarrow \mathbf{M} \circ \mathbf{W}_1, \quad \mathbf{M} = \text{np.random.binomial}(1, p, \text{size}=(n_2, n_1)),$$

also the bias is treated by dropping randomly entries.

References I

- [1] John Duchi, Elad Hazan, and Yoram Singer. “Adaptive Subgradient Methods for Online Learning and Stochastic Optimization”. In: *Journal of Machine Learning Research* 12 (July 2011), pp. 2121–2159. URL: <https://www.jmlr.org/papers/volume12/duchilla/duchilla.pdf>.
- [2] Alan Edelman and N. Raj Rao. “Random matrix theory”. In: *Acta Numer.* 14 (2005), pp. 233–297. ISSN: 0962-4929. DOI: 10.1017/S0962492904000236. URL: <https://doi.org/10.1017/S0962492904000236>.
- [3] Xavier Glorot and Yoshua Bengio. “Understanding the difficulty of training deep feedforward neural networks”. In: *Proceedings of Machine Learning Research* 9 (2010), pp. 249–256. URL: <http://proceedings.mlr.press/v9/glorot10a.html>.
- [4] Kaiming He et al. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”. In: *ICCV ’15: Proceedings of the 2015 IEEE International Conference on Computer Vision (ICCV)*. IEEE Computer Society Washington, DC, USA, Dec. 2015, pp. 1026–1034. ISBN: 978-1-4673-8391-2. DOI: 10.1109/ICCV.2015.123.

References II

- [5] Geoffrey Hinton. *Lecture 6e: rmsprop: Divide the gradient by a running average of its recent magnitude*. Slides of CSC321, Winter 2014. 2014. URL: http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.
- [6] Geoffrey E. Hinton et al. “Improving neural networks by preventing co-adaptation of feature detectors”. In: *arXiv e-prints*, arXiv:1207.0580 (July 2012). arXiv: 1207.0580 [cs.NE]. URL: <http://www.cs.toronto.edu/~hinton/absps/dropout.pdf>.
- [7] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *arXiv e-prints*, arXiv:1412.6980v9 (Jan. 2017). arXiv: 1412.6980v9 [cs.LG].
- [8] Madan Lal Mehta. *Random matrices*. Third. Vol. 142. Pure and Applied Mathematics (Amsterdam). Elsevier/Academic Press, Amsterdam, 2004, pp. xviii+688. ISBN: 0-12-088409-7.
- [9] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. “On the Convergence of Adam and Beyond”. In: *International Conference on Learning Representations*. 2018. URL: <https://openreview.net/forum?id=ryQu7f-RZ>.

References III

- [10] Andrew M. Saxe, James L. McClelland, and Surya Ganguli. “Exact solutions to the nonlinear dynamics of learning in deep linear neural networks”. In: *arXiv e-prints*, arXiv:1312.6120 (Dec. 2013). arXiv: 1312.6120 [cs.NE]. URL: https://openreview.net/forum?id=_wzZwKpTDF_9C.
- [11] Nitish Srivastava et al. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.56 (June 2014), pp. 1929–1958. URL: <https://www.jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf>.
- [12] Li Wan et al. “Regularization of Neural Networks using DropConnect”. In: *Proceedings of Machine Learning Research* 28.3 (2013). Proceedings of the 30th International Conference on Machine Learning, pp. 1058–1066. URL: <http://proceedings.mlr.press/v28/wan13.pdf>.
- [13] Matthew D. Zeiler. “ADADELTA: An Adaptive Learning Rate Method”. In: *arXiv e-prints*, arXiv:1212.5701 (Dec. 2012). arXiv: 1212.5701 [cs.LG].